



오늘 실습 목표

CSV 파일을 읽고 DataFrame을 생성하여
기본적인 데이터 탐색을 수행해봅시다!



준비물

- ✓ Google Colab (웹 브라우저)
- ✓ student_master.csv 파일
- ✓ Pandas 라이브러리



실습 목표 (학습 후 할 수 있어요!)

- 1 CSV 파일을 읽을 수 있다.
- 2 DataFrame을 생성하고 구조를 확인할 수 있다.
- 3 원하는 열(Column)을 선택할 수 있다.
- 4 조건에 맞는 데이터를 추출할 수 있다.
- 5 데이터를 정렬하고 분석할 수 있다.



실습 흐름



이번 실습에서는 실제 데이터를 활용하여
Pandas와 DataFrame의 기본 사용법을 직접 익혀봅시다!

★ 지금부터 본격적으로 데이터를 다루는 연습을 시작해봅시다! 😊





CSV 파일 읽기

: student_master.csv



실습 목표

CSV 파일을 읽어 DataFrame으로 불러오고 데이터를 확인해봅시다.



사용할 파일

student_master.csv

- 현재 폴더(Colab) 또는 업로드한 위치에 파일이 있어야 합니다.
- 파일 인코딩: UTF-8

1 CSV 파일 읽기

```
import pandas as pd

# CSV 파일을 DataFrame으로 읽기
df = pd.read_csv('student_master.csv')
```



TIP

파일 이름이 다르거나 위치가 다른 파일을 찾을 수 없습니다.

2 데이터 확인하기

① 상위 5개 행 확인

```
df.head()
```

② 기본 정보 확인

```
df.info()
```

③ 데이터 크기 확인

```
df.shape
```

	이름	성별	학년	학과	지역	동아리	출석률	국어	영어	수학	...
0	김민수	남	2	컴퓨터공학	서울	축구	95	85	88	90	...
1	이서연	여	1	데이터사이언스	경기	독서	92	90	92	89	...
2	박지훈	남	3	AI융합	인천	코딩	88	78	82	85	...
3	최유나	여	2	인공지능	부산	농구	97	93	95	94	...
4	정현우	남	1	컴퓨터공학	대전	코딩	90	84	87	83	...

5 rows x 14 columns

전체 흐름



CSV 파일 준비
(student_master.csv)



Colab에 파일 업로드
(또는 드라이브 연결)



pd.read_csv()
로 파일 읽기



DataFrame 생성
(df)



데이터 확인
(head, info, shape)

확인 포인트

- ✓ 파일을 올바르게 읽었는가?
- ✓ DataFrame(df)이 잘 생성되었는가?
- ✓ 행(Row)과 열(Column) 수를 확인했는가?
- ✓ 컬럼 이름과 데이터 타입을 확인했는가?



교수자 팁

파일이 보이지 않는다면? 왼쪽 파일 아이콘을 클릭하여 업로드하거나, 드라이브를 마운트하여 불러옵니다.





원하는 열(Columns) 선택하기



목표

DataFrame에서 특정 열(컬럼)만 선택하여 데이터를 확인해봅시다.

1 단일 열 선택

```
# 국어 점수 열 하나만 선택
df['국어']
```

실행 결과 (Series)

```
0    85
1    90
2    78
3    93
4    84
...
Name: 국어, Length: 50, dtype: int64
```

2 여러 열 선택

```
# 이름, 학과, 출석률 열 선택
df[['이름', '학과', '출석률']]
```

실행 결과 (DataFrame)

	이름	학과	출석률
0	김민수	컴퓨터공학	95
1	이서연	데이터사이언스	92
2	박지훈	AI융합	88
3	최유나	인공지능	97
4	정현우	컴퓨터공학	90
...	...	...	...

50 rows × 3 columns

3 연속된 여러 열 선택 (슬라이싱)

```
# 국어부터 Python까지 선택
df.loc[:, '국어': 'Python']
```

	국어	영어	수학	Python
0	85	88	90	92
1	90	92	89	92
2	78	82	85	88
3	93	95	94	97
4	84	87	83	90
...	...	...	...	...

50 rows × 4 columns



우리 데이터(student_master.csv)

	이름	성별	학년	학과	지역	동아리	출석률	국어	영어	수학	Python	데이터분석	진로희망	...
0	김민수	남	2	컴퓨터공학	서울	축구	95	85	88	90	92	90	개발자	...
1	이서연	여	1	데이터사이언스	경기	독서	92	90	92	89	96	94	데이터분석가	...
2	박지훈	남	3	AI융합	인천	코딩	88	78	82	85	88	86	AI엔지니어	...
3	최유나	여	2	인공지능	부산	농구	97	93	95	94	97	95	AI엔지니어	...
4	정현우	남	1	컴퓨터공학	대전	코딩	90	84	87	83	90	89	개발자	...
...	50 rows × 14 columns													



자주 사용하는 열 선택 방법

단일 열

df['컬럼명']

▶ 하나의 Series 반환

여러 열 (리스트)

df[['컬럼1', '컬럼2', ...]]

▶ DataFrame 반환

연속된 열 (슬라이싱)

df.loc[:, '시작열' : '끝열']

▶ 시작열부터 끝열까지

특정 순서 열 선택

df[['컬럼3', '컬럼1', '컬럼5']]

▶ 원하는 순서대로 선택



TIP

- 열 이름은 대소문자와 띄어쓰기까지 정확하게 입력해야 합니다.
- 열 이름이 여러 단어일 경우 언더스코어(_)로 연결되어 있습니다.
예) 데이터분석, 출석률, 진로희망



핵심 포인트

대괄호 [] 를 사용하여 열을 선택하며, 하나이면 Series, 여러 개이면 DataFrame이 반환됩니다.





열 선택하기

: 원하는 **Column**만 가져오기



실습 목표

DataFrame에서 원하는 열(Column)을 선택하여 분석에 필요한 데이터만 추출해봅시다.



핵심 개념

- DataFrame의 열(Column)은 이름으로 접근합니다.
- 하나의 열을 선택하면 Series가 반환됩니다.
- 여러 개의 열을 선택하면 DataFrame이 반환됩니다.

3 특정 순서로 열 선택

원하는 순서로 열 선택

```
cols = df[['수학', '영어', '이름']]
cols.head()
```

실행 결과 (DataFrame)

	수학	영어	이름
0	90	88	김민수
1	89	90	이서연
2	85	78	박지훈
3	94	93	최유나
4	83	84	정현우

4 여러 열을 변수로 저장하여 활용

분석에 필요한 열만 선택

```
analysis_df = df[['이름', '학과', '출석률', 'Python', '데이터분석']]
analysis_df.head()
```

실행 결과 (DataFrame)

	이름	학과	출석률	Python	데이터분석
0	김민수	컴퓨터공학	95	92	90
1	이서연	데이터사이언스	92	94	91
2	박지훈	AI융합	88	85	86
3	최유나	인공지능	97	96	93
4	정현우	컴퓨터공학	90	89	87



주의 사항

- ✓ 열 이름은 정확하게 입력해야 합니다.
- ✓ 대소문자를 구분합니다. (예: '국어' ≠ '국어')
- ✓ 존재하지 않는 열을 선택하면 KeyError가 발생합니다.



교수자 팁

열 선택은 데이터 전처리의 기본입니다.
필요한 열만 추출하면 코드가 간결해지고, 분석 속도도 빨라집니다!

1 하나의 열 선택 (Series 반환)

국어 점수 열 선택

```
kor = df['국어']
kor.head()
```

실행 결과 (Series)

```
0    85
1    92
2    88
3    97
4    90
Name: 국어, dtype: int64
```

2 여러 개의 열 선택 (DataFrame 반환)

이름, 국어, 수학 열 선택

```
subset = df[['이름', '국어', '수학']]
subset.head()
```

실행 결과 (DataFrame)

	이름	국어	수학
0	김민수	85	90
1	이서연	92	89
2	박지훈	88	85
3	최유나	97	94
4	정현우	90	83





학습 목표

DataFrame에서 하나의 열 또는 여러 개의 열을 선택하는 방법을 익혀봅니다.
student_master.csv 파일을 활용하여 아래 문제를 풀어보세요.

문제 1 영어 점수 열만 선택하여 출력하시오.

코드 입력

???

▶ 출력 예시 : Series 형태로 영어 점수가 출력됩니다.

문제 2 이름과 학과 열만 선택하여 출력하시오.

코드 입력

???

▶ 출력 예시 : DataFrame 형태로 이름과 학과가 출력됩니다.

문제 3 이름, Python, 데이터분석 열만 선택하여 출력하시오.

코드 입력

???

▶ 출력 예시 : DataFrame 형태로 선택한 3개 열이 출력됩니다.

도전 문제 국어, 영어, 수학 열만 선택하여 새로운 DataFrame을 생성하시오.

코드 입력

???

▶ 출력 예시 : 새로운 DataFrame으로 3개 과목만 포함됩니다.



힌트 (Hint)

하나의 열 선택

- `df['열이름']`

여러 개의 열 선택

- `df[['열이름1', '열이름2']]`

새로운 DataFrame 생성

- 선택한 열들을 이용하여 새로운 DataFrame을 만들 수 있습니다.

현재 데이터 컬럼 (예시)

이름	학번	학년	학과	성별
국어	영어	수학	Python	데이터분석
출석률	프로젝트	총점	평균	등급



실습 시간 : 5분



혼자서 코드를 작성해 보세요!

모든 문제를 풀어난 후 결과를 확인해 봅니다.



체크 포인트

- ✓ 하나의 열 선택이 가능하다.
- ✓ 여러 개의 열 선택이 가능하다.
- ✓ 선택한 열로 새로운 DataFrame을 생성할 수 있다.





행 선택하기 (Row Selection)

: 조건을 이용하여 원하는 행만 추출하기



실습 목표

조건(필터)을 사용하여 DataFrame에서 필요한 행만 선택해 봅시다.



핵심 개념

- 조건식은 True / False 를 반환합니다.
- 대괄호 [] 안에 조건식을 넣으면 True 인 행만 선택됩니다.
- 여러 조건을 조합할 수도 있습니다. (&, |, ~ 연산자)



우리 데이터 (student_master.csv)

	이름	성별	학년	학과	지역	동아리	출석률	국어	영어	수학	Python	데이터분석	진로희망	...
0	김민수	남	2	컴퓨터공학	서울	축구	95	85	88	90	92	90	개발자	...
1	이서연	여	1	데이터사이언스	경기	독서	92	90	92	89	96	94	데이터분석가	...
2	박지훈	남	3	AI융합	인천	코딩	88	78	82	85	88	86	AI엔지니어	...
3	최유나	여	2	인공지능	부산	농구	97	93	95	94	97	95	AI엔지니어	...
4	정현우	남	1	컴퓨터공학	대전	코딩	90	84	87	83	90	89	개발자	...
...	50 rows × 14 columns													

1 단일 조건으로 행 선택

```
# 수학 점수가 90점 이상인 학생
df[df['수학'] >= 90].head()
```

	이름	수학	Python	학과	지역
0	김민수	90	92	컴퓨터공학	서울
1	이서연	92	96	데이터사이언스	경기
3	최유나	95	97	인공지능	부산
6	한지민	91	94	데이터사이언스	서울
8	김태현	90	92	컴퓨터공학	대구

5 rows × 14 columns

★ 수학 점수 >= 90 인 행만 선택

2 여러 조건으로 행 선택 (AND)

```
# 출석률이 90 이상이고, 국어가 85 이상
df[(df['출석률'] >= 90) &
    (df['국어'] >= 85)].head()
```

	이름	출석률	국어	학과	지역
0	김민수	95	85	컴퓨터공학	서울
1	이서연	92	90	데이터사이언스	경기
3	최유나	97	93	인공지능	부산
5	오수빈	91	86	AI융합	광주
6	한지민	94	91	데이터사이언스	서울

5 rows × 14 columns

★ 두 조건을 모두 만족하는 행만 선택

3 여러 조건으로 행 선택 (OR)

```
# 학과가 'AI융합' 또는 '인공지능'
df[(df['학과'] == 'AI융합') |
    (df['학과'] == '인공지능')].head()
```

	이름	학과	지역	Python	데이터분석
2	박지훈	AI융합	인천	88	86
4	정현우	컴퓨터공학	대전	90	89
5	오수빈	AI융합	광주	91	90
3	최유나	인공지능	부산	97	95
9	이지훈	인공지능	울산	93	92

5 rows × 14 columns

★ 두 조건 중 하나라도 만족하는 행 선택

4 NOT 조건으로 행 선택

```
# 출석률이 90 미만인 학생
df[~(df['출석률'] >= 90)].head()
```

	이름	출석률	학과	지역
2	박지훈	88	AI융합	인천
7	강민지	89	컴퓨터공학	대구
8	김태현	87	컴퓨터공학	대구
10	유승호	85	데이터사이언스	수원
11	황서연	88	AI융합	전주

5 rows × 14 columns

★ 조건을 만족하지 않는 행 선택



실습 팁

- ✓ 비교 연산자 : >, <, >=, <=, ==, !=
- ✓ 논리 연산자 : &, |, ~ (괄호를 사용하면 더 정확합니다.)
- ✓ 문자열 비교는 큰따옴표 또는 작은따옴표 사용
예) df[df['성별'] == '여']



주의 사항

- 조건식 결과는 True / False 입니다.
- 컬럼 이름을 정확히 입력해야 합니다.
- 자료형이 숫자인 열만 비교 연산이 가능합니다.
(문자열 비교는 ==, != 만 사용)



다음 단계 미리보기

선택한 데이터를 정렬하거나,
새로운 열을 만들어 분석을 더
깊이 진행해 봅시다!





학습 목표

DataFrame에서 특정 행을 선택하는 방법을 익혀봅니다.
student_master.csv 파일을 활용하여 아래 문제를 풀어보세요.

문제 1 처음 10개 행을 출력하시오.



힌트

DataFrame의 앞쪽 일부 행을 확인할 때 사용하는 함수입니다.

코드 입력

???

▶ 예상 결과 (일부)

	이름	학번	학년	학과	성별	국어	영어	...
0	김민수	2024001	2	컴퓨터공학	남	85	90	...
1	이서연	2024002	1	데이터사이언스	여	92	88	...
...	...	...	...	...	...	...	...	...
9	박지훈	20240010	3	AI융합	남	78	82	...

▶ 출력 예시 : DataFrame 행태로 10개 행이 출력됩니다.

문제 2 5번째부터 10번째까지의 행을 출력하시오.



힌트

iloc[]를 사용하면 원하는 위치의 행을 선택할 수 있습니다.

코드 입력

???

▶ 예상 결과 (일부)

	이름	학번	학년	학과	성별	국어	영어	...
4	정현우	2024005	2	인공지능	남	95	93	...
5	최유나	2024006	2	인공지능	여	88	90	...
...	...	...	...	...	...	...	...	...
9	박지훈	20240010	3	AI융합	남	78	82	...

▶ 출력 예시 : 5번째부터 10번째 행까지 출력됩니다.

문제 3 처음 3개 행만 출력하시오.



힌트

head() 함수를 사용하면 기본값은 5개지만, 괄호 안에 숫자를 지정할 수 있습니다.

코드 입력

???

▶ 예상 결과

	이름	학번	학년	학과	성별	국어	영어	...
0	김민수	2024001	2	컴퓨터공학	남	85	90	...
1	이서연	2024002	1	데이터사이언스	여	92	88	...
2	박지훈	2024003	3	AI융합	남	78	82	...

▶ 출력 예시 : 처음 3개 행이 출력됩니다.

★ 사용 가능한 방법

- head(n) : 앞에서부터 n개 행 선택
- iloc[start:end] : 인덱스 위치 기반 선택 (start는 포함, end는 미포함)
- iloc[start:] : start부터 끝까지 선택
- iloc[:end] : 처음부터 end-1까지 선택

★ 현재 데이터 미리보기

인덱스	이름	학번	학년	학과	성별	...
0	김민수	2024001	2	컴퓨터공학	남	...
1	이서연	2024002	1	데이터사이언스	여	...
2	박지훈	2024003	3	AI융합	남	...
...	...	...	...	...	...	...
107	장현우	2024108	1	컴퓨터공학	남	...
108	이수민	2024109	2	데이터사이언스	여	...
109	한지민	2024110	1	AI융합	여	...



총 110개의 행(학생)과 여러 개의 열(컬럼)로 구성된 데이터입니다.



실습 시간 : 5분

문제를 풀고 결과를 확인해 보세요!



혼자서 해보세요!

각 문제를 코드로 작성한 후 실행하여 결과를 확인해 보세요.



체크 포인트

- ✓ head() 함수로 앞쪽 행을 선택할 수 있다.
- ✓ iloc[]를 사용하여 원하는 위치의 행을 선택할 수 있다.
- ✓ 슬라이싱을 이용하여 행 범위를 지정할 수 있다.





행 필터링 (Row Filtering)

: 조건을 이용하여 원하는 데이터만 추출하기



실습 목표

비교 연산자와 논리 연산자를 이용하여
조건에 맞는 행(Row)만 추출해 봅시다.



사용 데이터: student_master.csv

	이름	성별	학년	학과	지역	동아리	출석률	국어	영어	수학	Python	데이터분석	진로희망	...
0	김민수	남	2	컴퓨터공학	서울	축구	95	85	88	90	92	90	개발자	...
1	이서연	여	1	데이터사이언스	경기	독서	92	90	92	89	96	94	데이터분석가	...
2	박지훈	남	3	AI융합	인천	코딩	88	78	82	85	88	86	AI엔지니어	...
3	최유나	여	2	인공지능	부산	농구	97	93	95	94	97	95	AI엔지니어	...
4	정현우	남	1	컴퓨터공학	대전	코딩	90	84	87	83	90	89	개발자	...
...	50 rows x 14 columns													

1 단일 조건 필터링

```
# 수학 점수가 90점 이상인 학생
df[df['수학'] >= 90].head()
```

	이름	수학	학과	...
0	김민수	90	컴퓨터공학	...
1	이서연	92	데이터사이언스	...
3	최유나	95	인공지능	...
5	오수빈	91	AI융합	...
9	이지훈	93	인공지능	...

5 rows x 14 columns

2 두 조건 AND (모두 만족)

```
# 출석률이 90 이상이고, 국어가 85 이상
df[(df['출석률'] >= 90) &
    (df['국어'] >= 85)].head()
```

	이름	출석률	국어	...
0	김민수	95	85	...
1	이서연	92	90	...
3	최유나	97	93	...
6	한지민	91	88	...
8	김태현	90	85	...

5 rows x 14 columns

3 두 조건 OR (하나라도 만족)

```
# 학과가 'AI융합' 또는 '인공지능'
df[(df['학과'] == 'AI융합') |
    (df['학과'] == '인공지능')].head()
```

	이름	학과	...
2	박지훈	AI융합	...
3	최유나	인공지능	...
5	오수빈	AI융합	...
6	한지민	인공지능	...
9	이지훈	인공지능	...

5 rows x 14 columns

4 AND + OR 조합

```
# (학년이 1학년이거나 2학년) 이고,
# Python 점수가 90 이상
df[((df['학년'] <= 2) & (df['학년'] >= 1)) &
    (df['Python'] >= 90)].head()
```

	이름	학년	Python	...
0	김민수	2	92	...
1	이서연	1	96	...
4	정현우	1	90	...
6	한지민	2	91	...

4 rows x 14 columns

5 NOT 조건 (부정)

```
# 출석률이 90 미만인 학생
df[~(df['출석률'] >= 90)].head()
```

	이름	출석률	...
2	박지훈	88	...
5	오수빈	89	...
7	강민지	87	...
10	유승호	85	...
12	장서연	86	...

5 rows x 14 columns

6 여러 조건 조합 (실전 예제)

```
# 컴퓨터공학 학과이고, 출석률 90 이상이며,
# Python 점수도 90 이상인 학생
df[(df['학과'] == '컴퓨터공학') &
    (df['출석률'] >= 90) &
    (df['Python'] >= 90)].head()
```

	이름	학과	출석률	Python	...
0	김민수	컴퓨터공학	95	92	...
4	정현우	컴퓨터공학	90	90	...

2 rows x 14 columns



실습 팁

- ✓ 조건을 하나씩 추가하며 결과를 확인하세요.
- ✓ 괄호 () 위치가 올바르게 확인하세요.
- ✓ 복잡한 조건은 여러 줄로 작성하면 가독성이 좋아집니다.



주의 사항

- 조건식에서 연산자 우선순위로 인해 오류가 날 수 있으므로 반드시 괄호 () 로 묶어주세요.
- & , | 은 and, or 가 아닙니다! (Pandas에서는 & , | 사용)



핵심 정리

필터링을 통해 분석에 필요한 데이터만 추출하면
분석의 정확도와 효율성이 높아집니다!





학습 목표

특정 조건을 만족하는 데이터를 추출하는 방법을 익혀봅니다.
student_master.csv 파일을 활용하여 아래 문제를 풀어보세요.

문제 1 수학 점수 90점 이상인 학생을 찾아보세요.



힌트

조건식은 df[조건] 형태로 작성합니다.

코드 입력

???

▶ 예상 결과 (일부)

	이름	학과	수학	국어	영어	...
2	박지훈	AI융합	95	78	82	...
8	한지원	AI융합	91	88	85	...
15	정현우	인공지능	95	93	90	...
...	...	...	...	...	...	...

▶ 출력 예시 : `df[df['수학'] >= 90]`

문제 2 출석률 95% 이상인 학생을 찾아보세요.



힌트

출석률은 '출석률' 컬럼입니다.

코드 입력

???

▶ 예상 결과 (일부)

	이름	학과	출석률	Python	데이터분석	...
4	최유나	인공지능	98	88	90	...
11	장현우	컴퓨터공학	97	91	92	...
15	이수민	데이터사이언스	96	82	85	...
...	...	...	...	...	...	...

▶ 출력 예시 : `df[df['출석률'] >= 95]`

문제 3 Python 점수 85점 이상인 학생을 찾아보세요.



힌트

Python 점수는 'Python' 컬럼입니다.

코드 입력

???

▶ 예상 결과 (일부)

	이름	학과	Python	데이터분석	출석률	...
1	이서연	데이터사이언스	92	88	92	...
7	김민수	컴퓨터공학	85	90	90	...
11	장현우	컴퓨터공학	91	92	97	...
...	...	...	...	...	...	...

▶ 출력 예시 : `df[df['Python'] >= 85]`

★ 다양한 조건식 예시

- `df[df['수학'] > 80]`
- `df[df['출석률'] >= 90]`
- `df[df['Python'] <= 70]`
- `df[(df['수학'] >= 90) & (df['출석률'] >= 95)]`

★ 현재 데이터 미리보기 (일부)

이름	학과	수학	Python	출석률	...
김민수	컴퓨터공학	85	85	90	...
이서연	데이터사이언스	88	92	92	...
박지훈	AI융합	95	78	88	...
최유나	인공지능	88	88	98	...
...	...	...	...	...	...



주의!

조건을 만족하는 행만 추출되며,
원본 DataFrame은 변경되지 않습니다.



실습 시간 : 5분

문제를 풀고 결과를 확인해 보세요!



혼자서 해보세요!

각 문제를 스스로 해결한 후
결과를 확인해 보세요.



체크 포인트

- ✓ 조건식을 이용해 원하는 데이터만 추출할 수 있다.
- ✓ 다양한 비교 연산자(>, <, >=, <=, ==, !=)를 사용할 수 있다.
- ✓ 여러 조건(& / |)을 조합하여 필터링할 수 있다.





정렬하기 (Sorting)

: 특정 열 기준으로 데이터를 정렬하여 원하는 순서로 확인하기



실습 목표

정렬(Sort)을 활용하여 데이터를
오름차순(asc) 또는 내림차순(desc)으로 정렬해 봅시다.



핵심 개념

- `sort_values()` 함수를 사용하여 정렬합니다.
- `ascending=True` : 오름차순(기본값)
- `ascending=False` : 내림차순
- 여러 열을 기준으로 정렬할 수도 있습니다.



우리 데이터: student_master.csv

	이름	성별	학년	학과	지역	동아리	출석률	국어	영어	수학	Python	데이터분석	진로희망	...
0	김민수	남	2	컴퓨터공학	서울	축구	95	85	88	90	92	90	개발자	...
1	이서연	여	1	데이터사이언스	경기	독서	92	90	92	89	96	94	데이터분석가	...
2	박지훈	남	3	AI융합	인천	코딩	88	78	82	85	88	86	AI엔지니어	...
3	최유나	여	2	인공지능	부산	농구	97	93	95	94	97	95	AI엔지니어	...
4	정현우	남	1	컴퓨터공학	대전	코딩	90	84	87	83	90	89	개발자	...

... 50 rows x 14 columns

1 수학 점수 기준 오름차순 정렬

```
df.sort_values(by='수학').head(5)
```

	이름	수학	학과
7	강민지	62	컴퓨터공학
18	정민수	65	데이터사이언스
23	한지민	68	AI융합
2	박지훈	82	AI융합
4	정현우	83	컴퓨터공학

5 rows x 14 columns

★ 수학 점수가 낮은 학생부터 정렬

2 수학 점수 기준 내림차순 정렬

```
df.sort_values(by='수학',  
               ascending=False).head(5)
```

	이름	수학	학과
12	이지훈	98	인공지능
16	김태현	97	AI융합
11	오수빈	96	AI융합
3	최유나	95	인공지능
25	배서연	95	컴퓨터공학

5 rows x 14 columns

★ 수학 점수가 높은 학생부터 정렬

3 국어, 영어 기준 오름차순 정렬

```
df.sort_values(by=['국어', '영어'],  
               .head(5))
```

	이름	국어	영어	학과
2	박지훈	78	82	AI융합
14	홍세영	79	83	데이터사이언스
18	정민수	80	85	AI융합
21	김도현	81	78	컴퓨터공학
7	강민지	82	85	컴퓨터공학

5 rows x 14 columns

★ 국어 점수 → 영어 점수 순으로 정렬

4 출석률 기준 내림차순 정렬

```
df.sort_values(by='출석률',  
               ascending=False).head(5)
```

	이름	출석률	학과
3	최유나	97	인공지능
44	이수현	96	컴퓨터공학
0	김민수	95	컴퓨터공학
17	박서연	95	데이터사이언스
29	전하운	94	AI융합

5 rows x 14 columns

★ 출석률이 높은 학생부터 정렬

5 두 기준으로 정렬 (수학 내림차순, 출석률 내림차순)

```
df.sort_values(by=['수학', '출석률'],  
               ascending=[False, False]).head(5)
```

	이름	수학	출석률	학과
12	이지훈	98	91	인공지능
16	김태현	97	94	AI융합
11	오수빈	96	92	AI융합
3	최유나	95	97	인공지능
25	배서연	95	90	컴퓨터공학

5 rows x 14 columns

★ 수학 내림차순 → 출석률 내림차순



실습 팁

- ✓ `ascending=True` 는 생략 가능합니다. (기본값)
- ✓ 여러 열을 기준으로 정렬하면 앞의 열이 우선순위가 됩니다.
- ✓ 정렬 후 원본 데이터가 변경되지 않습니다.
- ✓ `reset_index(drop=True)`를 사용하면 인덱스를 다시 정렬할 수 있습니다.

```
df_sorted = df.sort_values(by='수학', ascending=False).reset_index(drop=True)
```



주의 사항

- 문자 열(이름, 학과 등)도 사전식으로 정렬됩니다.
- 결측치(NaN)는 기본적으로 마지막에 위치합니다.

예시) 이름 기준 정렬

```
df.sort_values(by='이름').head(5)
```

★ 핵심 정리

정렬(Sort)은 데이터를 원하는 순서로 보이게 하여
분석과 비교를 쉽게 만들어 줍니다.
분석 전 데이터 정리는 데이터 분석의 기본입니다!



교수자 팁

정렬 결과를 통해 상위/하위 학생, 성적 순위, 출석 우수 학생 등 다양한 인사이트를 도출할 수 있습니다.



학습 목표

DataFrame을 특정 열을 기준으로 오름차순/내림차순 정렬하는 방법을 익혀봅니다.
student_master.csv 파일을 활용하여 아래 문제를 풀어보세요.

문제 1 국어 점수 기준으로 내림차순 정렬하세요.



힌트

sort_values()의 ascending=False를 사용합니다.

코드 입력

???

▶ 예상 결과 (상위 5개)

이름	학과	국어	영어	수학	...
박지훈	AI융합	95	88	92	...
정현우	컴퓨터공학	93	90	88	...
최유나	인공지능	92	88	90	...
이수민	데이터사이언스	90	92	85	...
한지민	AI융합	89	85	86	...
...	...	...	...	...	...

▶ 출력 예시 : `df.sort_values('국어', ascending=False)`

문제 2 출석을 기준으로 오름차순 정렬하세요.



힌트

ascending=True (기본값) 또는 생략 가능합니다.

코드 입력

???

▶ 예상 결과 (상위 5개)

이름	학과	출석률	국어	영어	...
김민수	컴퓨터공학	78	85	90	...
박지훈	AI융합	82	95	88	...
장현우	컴퓨터공학	90	93	90	...
이수민	데이터사이언스	95	90	92	...
최유나	인공지능	98	92	88	...
...	...	...	...	...	...

▶ 출력 예시 : `df.sort_values('출석률', ascending=True)`

문제 3 Python 점수 기준으로 내림차순 정렬 후 상위 5명의 이름과 Python 점수를 출력하세요.



힌트

정렬 후 head(5)를 사용하세요.

코드 입력

???

▶ 예상 결과 (상위 5명)

이름	Python
박지훈	95
장현우	91
이수민	90
최유나	88
정현우	88

▶ 출력 예시 : `df.sort_values('Python', ascending=False).head(5)['이름', 'Python']`

★ 사용 방법

- `sort_values(by='열이름', ascending=True/False)`
- `ascending=False` : 내림차순 (큰값 → 작은값)
- `ascending=True` : 오름차순 (작은값 → 큰값)
- 기본값은 `ascending=True` (오름차순)

★ 현재 데이터 미리보기 (일부)

이름	학과	국어	영어	수학	Python	출석률
김민수	컴퓨터공학	85	90	88	85	78
이서연	데이터사이언스	88	92	85	88	95
박지훈	AI융합	95	88	92	95	82
...	...	...	...	...	...	...
장현우	컴퓨터공학	93	90	88	91	90
이수민	데이터사이언스	90	92	85	90	95
한지민	AI융합	89	85	86	78	88
...	...	...	...	...	...	...



도전 문제

국어 점수가 같을 경우, Python 점수를 기준으로 내림차순 정렬해 보세요!

(힌트: `by=['국어', 'Python'], ascending=[False, False]`)



실습 시간 : 6분

주어진 시간 안에 문제를 풀어보세요!



혼자서 해결해 보세요!

코드를 작성하고 결과를 확인한 후,
옆 친구와 서로 비교해 보세요.



체크 포인트

- ✓ 원하는 열을 기준으로 정렬할 수 있다.
- ✓ 오름차순과 내림차순을 구분하여 정렬할 수 있다.
- ✓ 정렬 후 head() 등을 활용하여 원하는 결과를 추출할 수 있다.





상관관계 분석 (Correlation Analysis)

: 두 변수 간의 상관관계를 계산하고 해석하기



실습 목표

수치형 변수들 간의 상관관계를 계산하고,
상관계수 행렬과 히트맵을 시각화하여 해석해 봅시다.



핵심 개념

- corr() : 상관계수 계산 (피어슨 상관계수, 기본값)
- 상관계수 범위 : -1 ~ 1
 - 1에 가까울수록 강한 양의 상관관계
 - 1에 가까울수록 강한 음의 상관관계
 - 0에 가까울수록 상관관계가 없음
- 수치형 열만 선택하여 상관분석을 수행합니다.

1 수치형 열 선택

```
# 수치형 열 선택
num_cols = ['출석률', '국어', '영어', '수학', 'Python', '데이터분석']
numeric_df = df[num_cols]
numeric_df.head()
```

	출석률	국어	영어	수학	Python	데이터분석
0	95	85	88	90	92	90
1	92	90	92	89	96	94
2	88	78	82	85	88	86
3	97	93	95	94	97	95
4	90	84	87	83	90	89

5 rows x 6 columns

2 상관계수 행렬 계산

```
# 상관계수 행렬 계산
corr_matrix = numeric_df.corr()
corr_matrix.round(2)
```

	출석률	국어	영어	수학	Python	데이터분석
출석률	1.00	0.62	0.58	0.64	0.72	0.75
국어	0.62	1.00	0.81	0.68	0.59	0.63
영어	0.58	0.81	1.00	0.69	0.57	0.61
수학	0.64	0.68	0.69	1.00	0.76	0.78
Python	0.72	0.59	0.57	0.76	1.00	0.92
데이터분석	0.75	0.63	0.61	0.78	0.92	1.00

6 rows x 6 columns

✓ Python과 데이터분석 점수의 상관계수가 0.92로 매우 높음!

우리 데이터: student_master.csv)

	이름	성별	학년	학과	지역	동아리	출석률	국어	영어	수학	Python	데이터분석	진로희망	...
0	김민수	남	2	컴퓨터공학	서울	축구	95	85	88	90	92	90	개발자	...
1	이서연	여	1	데이터사이언스	경기	독서	92	90	92	89	96	94	데이터분석가	...
2	박지훈	남	3	AI융합	인천	코딩	88	78	82	85	88	86	AI엔지니어	...
3	최유나	여	2	인공지능	부산	농구	97	93	95	94	97	95	AI엔지니어	...
4	정현우	남	1	컴퓨터공학	대전	코딩	90	84	87	83	90	89	개발자	...

... 50 rows x 14 columns

★ 분석에 사용할 수치형 열

출석률, 국어, 영어, 수학, Python, 데이터분석
(총 6개 열)

3 상관계수 히트맵 시각화

```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(7,5))
sns.heatmap(corr_matrix, annot=True,
            cmap='RdBu_r', center=0, fmt='.2f')
plt.title('수치형 변수 상관관계 히트맵')
plt.tight_layout()
plt.show()
```

수치형 변수 상관관계 히트맵



4 상관관계 해석 예시

- ✓ Python ↔ 데이터분석 : 0.92
→ 매우 강한 양의 상관관계
- ✓ 수학 ↔ Python : 0.76
→ 강한 양의 상관관계
- ✓ 국어 ↔ 영어 : 0.81
→ 높은 양의 상관관계
- ✓ 출석률 ↔ 모든 과목
→ 0.58 ~ 0.75 사이로
중간~높은 양의 상관관계



TIP

상관관계는 인과관계를 의미하지 않습니다.
두 변수가 함께 증가/감소하는 경향만
나타냅니다!



실습 팁

- ✓ 수치형 열만 선택해야 corr() 함수가 작동합니다.
- ✓ 결측치가 있으면 상관계수가 NaN이 될 수 있습니다.
- ✓ seaborn 추가 설치 필요 (pip install seaborn)



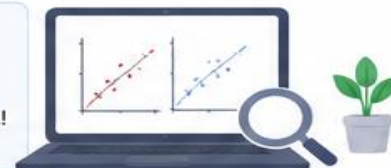
주의 사항

- 상관계수가 높다고 해서 관계가 있다고 단정할 수 없습니다.
- 데이터의 분포와 이상치에 따라 결과가 달라질 수 있습니다.
- 결측치가 많으면 결과 해석에 주의해야 합니다.



교수자 팁

상관분석 결과를 바탕으로 가설을 세우고,
그 다음 단계에서 회귀분석 등 심화 분석으로 확장해 보세요!





그룹별 집계와 평균 비교 (GroupBy & Aggregation)

: 그룹별로 데이터를 집계하고, 평균을 비교하여 해석하기



실습 목표

groupby()를 이용해 범주별 평균, 합계, 개수 등을 계산하고, 결과를 정렬하여 비교·해석할 수 있습니다.



핵심 개념

- groupby('열') : 특정 열을 기준으로 그룹화
- 집계 함수 : mean(), sum(), count(), max(), min() 등
- agg() : 여러 집계 함수를 한 번에 적용
- 정렬 : sort_values()로 결과를 정렬하여 비교

★ 이번 실습에서 사용할 열

학년, 학과, 성별, 지역

수치형 열: 출석률, 국어, 영어, 수학, Python, 데이터분석

우리 데이터: student_master.csv

	이름	성별	학년	학과	지역	동아리	출석률	국어	영어	수학	Python	데이터분석	진로희망	...
0	김민수	남	2	컴퓨터공학	서울	축구	95	85	88	90	92	90	개발자	...
1	이서연	여	1	데이터사이언스	경기	독서	92	90	92	89	96	94	데이터분석가	...
2	박지훈	남	3	AI융합	인천	코딩	88	78	82	85	88	86	AI엔지니어	...
3	최유나	여	2	인공지능	부산	농구	97	93	95	94	97	95	AI엔지니어	...
4	정현우	남	1	컴퓨터공학	대전	코딩	90	84	87	83	90	89	개발자	...
...	50 rows × 14 columns													

1 학년별 평균 점수 비교

학년별 각 과목의 평균

```
grade_mean = df.groupby('학년')[['국어', '영어', '수학', 'Python', '데이터분석']].mean().round(2)
grade_mean
```

학년	국어	영어	수학	Python	데이터분석
1	83.40	84.80	85.60	86.20	87.00
2	86.38	87.31	88.08	89.15	89.85
3	82.75	83.25	83.81	84.25	85.06
4	84.25	85.13	86.00	86.88	87.50

★ 2학년 학생들의 전반적인 성적이 가장 높음!

2 학과별 Python 평균 비교 (정렬)

학과별 Python 평균 내림차순 정렬

```
dept_python = df.groupby('학과')['Python'].mean().sort_values(ascending=False).round(2)
dept_python
```

학과	Python 평균
AI융합	92.40
인공지능	90.83
데이터사이언스	89.15
컴퓨터공학	87.78

★ AI융합 전공의 Python 평균이 가장 높음!

3 성별별 평균 비교

성별별 각 과목 평균

```
gender_mean = df.groupby('성별')[['국어', '영어', '수학', 'Python', '데이터분석']].mean().round(2)
gender_mean
```

성별	국어	영어	수학	Python	데이터분석
남	84.40	85.20	86.00	87.60	88.50
여	85.30	86.10	86.90	88.70	89.70

★ 여학생의 평균 점수가 전 과목에서 조금 더 높음!

4 지역별 출석률을 평균과 학생 수

지역별 출석률 평균과 학생 수

```
region_agg = df.groupby('지역')['출석률'].agg(['mean', 'count']).round(2)
region_agg
```

지역	mean	count
서울	94.10	10
경기	92.60	12
인천	91.80	7
부산	93.00	8
대전	92.40	6
광주	92.20	4
울산	91.50	3

★ 서울 학생들의 출석률이 가장 높음!

5 동아리별 인원수 Top 5

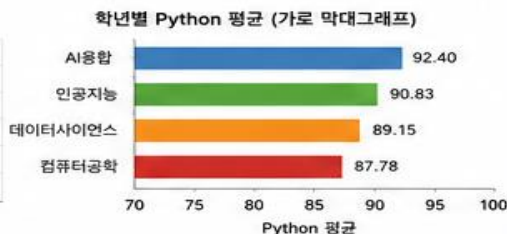
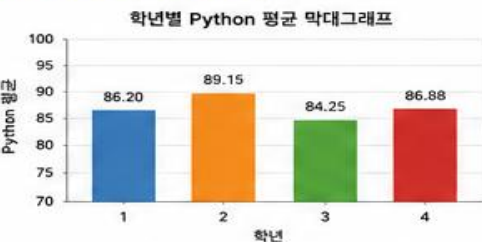
동아리별 인원수 Top 5

```
club_top = df['동아리'].value_counts().head(5)
club_top
```

동아리	인원수
코딩	11
독서	9
축구	8
봉사	6
농구	5

★ 코딩 동아리 인원이 가장 많음!

추가 시각화 (예시)



⚠ 주의 사항

- groupby()의 결과는 인덱스(그룹명)가 됩니다. 필요시 reset_index()를 사용하세요.
- 평균 비교 시 반올림(round())을 사용하면 보기 좋습니다.
- 정렬(sort_values) 시 ascending=True/False를 확인하세요.
- 결측치(NaN)가 있는 열은 평균 계산 시 자동으로 제외됩니다.

💡 실습 팁

- ✓ 다양한 열 조합으로 groupby()를 시도해 보세요. (예: 학년+성별, 지역+학과 등)
- ✓ agg()에 여러 함수를 넣어 비교해 보세요. 예).agg(['mean', 'max', 'min', 'count'])
- ✓ 결과를 시각화하면 해석이 더 쉽고 효과적입니다!





학습 목표

DataFrame을 특정 열 기준으로 그룹화하여 평균을 계산하는 방법을 익혀봅시다.
student_master.csv 파일을 활용하여 아래 문제를 풀어보세요.

문제 1 학년별 평균 출석률을 계산하세요.



힌트

groupby('학년') 후 평균을 계산합니다.

코드 입력

???

▶ 예상 결과 (참고)

학년	출석률(평균)
1	92.1
2	93.4
3	91.8
4	90.6

▶ 출력 예시 : `df.groupby('학년')['출석률'].mean()`

문제 2 성별 평균 Python 점수를 계산하세요.



힌트

groupby('성별') 후 Python 점수 평균을 계산합니다.

코드 입력

???

▶ 예상 결과 (참고)

성별	Python(평균)
남	88.7
여	89.9

▶ 출력 예시 : `df.groupby('성별')['Python'].mean()`

문제 3 학과별 평균 데이터분석 점수를 계산하세요.



힌트

groupby('학과') 후 데이터분석 평균을 계산합니다.

코드 입력

???

▶ 예상 결과 (참고)

학과	데이터분석(평균)
AI융합	86.3
데이터사이언스	88.7
컴퓨터공학	87.9
인공지능	89.1

▶ 출력 예시 : `df.groupby('학과')['데이터분석'].mean()`

★ GroupBy 주요 문법

```
df.groupby('열').mean()
```

지정한 열을 기준으로 그룹화하여
평균을 계산합니다.

- 여러 열을 선택하려면
`df.groupby('열')[['열1', '열2']].mean()`
- 다른 집계 함수도 사용할 수 있습니다.
(sum, count, min, max 등)

★ 현재 데이터 미리보기 (일부)

이름	학년	학과	성별	Python	데이터분석	출석률
김민수	2	컴퓨터공학	남	85	78	90
이서연	1	데이터사이언스	여	92	88	95
박지훈	3	AI융합	남	95	82	88
최유나	2	인공지능	여	88	90	98
...	...	...	...	...	...	...



주의!

GroupBy 결과는 자동으로 정렬됩니다.
원본 데이터는 변경되지 않습니다.



실습 시간 : 7분

문제를 풀고 결과를 확인해 보세요!



혼자서 해보세요!

각 문제를 스스로 해결한 후
결과를 확인하고 정리해 보세요.



체크 포인트

- groupby()로 데이터를 그룹화할 수 있다.
- 원하는 열의 평균(또는 다른 집계)을 계산할 수 있다.
- 결과를 해석하고 비교할 수 있다.





결측치 처리 (Missing Values)

: 결측치(NaN)를 확인하고 다양한 방법으로 처리하기



실습 목표

데이터의 결측치를 확인하고, 제거하거나 대체하는 방법을 실습하여 데이터 전처리의 중요성을 이해합니다.



핵심 개념

- `isna()` / `notna()` : 결측치 확인
- `dropna()` : 결측치가 포함된 행/열 제거
- `fillna()` : 결측치를 특정 값, 평균/중앙값 등으로 대체
- 결측치 처리는 분석 결과의 정확도에 큰 영향을 줍니다.



우리 데이터: student_master_missing.csv

	이름	성별	학년	학과	지역	동아리	출석률	국어	영어	수학	Python	데이터분석	진로희망	...
0	김민수	남	2	컴퓨터공학	서울	축구	95.0	85.0	88.0	90.0	92.0	90.0	개발자	...
1	이서연	여	1	데이터사이언스	경기	독서	92.0	90.0	92.0	89.0	96.0	94.0	데이터분석가	...
2	박지훈	남	3	AI융합	인천	코딩	NaN	78.0	82.0	85.0	88.0	NaN	AI엔지니어	...
3	최유나	여	2	인공지능	부산	농구	97.0	NaN	95.0	94.0	97.0	95.0	AI엔지니어	...
4	정현우	남	1	컴퓨터공학	대전	코딩	90.0	84.0	NaN	83.0	NaN	89.0	개발자	...

... 50 rows × 14 columns (NaN은 결측치)

★ 이번 실습에서 사용할 열 : 출석률, 국어, 영어, 수학, Python, 데이터분석 (6개 열)

1 결측치 확인

```
# 각 열의 결측치 개수 확인
df.isna().sum()
```

열 이름	결측치 개수
출석률	2
국어	1
영어	2
수학	0
Python	2
데이터분석	1

★ 데이터를 다루기 전에 결측치를 반드시 확인!

2 결측치가 있는 행 제거 (dropna)

```
df_drop_row = df.dropna()
df_drop_row.shape
```

(46, 14)

결과 확인

	이름	출석률	국어	영어	수학	Python	데이터분석
0	김민수	95.0	85.0	88.0	90.0	92.0	90.0
1	이서연	92.0	90.0	92.0	89.0	96.0	94.0
5	오수빈	91.0	86.0	90.0	91.0	91.0	90.0
6	한지민	94.0	91.0	89.0	92.0	93.0	92.0
...	...	...	...	...	...	...	...

46 rows × 14 columns

★ 결측치가 하나라도 있는 행을 모두 제거

3 결측치가 있는 열 제거 (dropna)

```
df_drop_col = df.dropna(axis=1)
df_drop_col.columns
```

```
Index(['이름', '성별', '학년', '학과', '지역', '동아리',  
      '수학', '진로희망', ...], dtype='object')
```

결과 확인

	이름	성별	학년	학과	지역	동아리	수학	진로희망
0	김민수	남	2	컴퓨터공학	서울	축구	90.0	개발자
1	이서연	여	1	데이터사이언스	경기	독서	89.0	데이터분석가
3	박지훈	남	3	AI융합	인천	코딩	85.0	AI엔지니어
3	최유나	여	2	인공지능	부산	농구	94.0	AI엔지니어
4	정현우	남	1	컴퓨터공학	대전	코딩	83.0	개발자
...	...	...	...	...	...	...	...	...

50 rows × 9 columns

★ 결측치가 하나라도 있는 열을 모두 제거

4 결측치 대체: 평균으로 채우기

```
# 수치형 열의 결측치를 평균으로 대체
num_cols = ['출석률', '국어', '영어', '수학', 'Python', '데이터분석']

df_fill_mean = df.copy()
df_fill_mean[num_cols] =  
df_fill_mean[num_cols].fillna(df[num_cols].mean())
```

결과 확인 (결측치 개수)

```
df_fill_mean[num_cols].isna().sum()
```

열 이름	결측치 개수
출석률	0
국어	0
영어	0
수학	0
Python	0
데이터분석	0

★ 평균으로 대체하여 모든 결측치를 채움

5 결측치 대체: 특정 값으로 채우기

```
# 결측치를 0으로 대체
df_fill_zero = df.copy()
df_fill_zero[num_cols] =  
df_fill_zero[num_cols].fillna(0)
```

결과 확인 (결측치 개수)

```
df_fill_zero[num_cols].isna().sum()
```

열 이름	결측치 개수
출석률	0
국어	0
영어	0
수학	0
Python	0
데이터분석	0

★ 특정 값(0)으로 결측치를 채움



실습 팁

- ✓ `isna()`는 True/False로 결측치를 표시합니다.
- ✓ `dropna()` 사용 시 `how='any'` (기본값), `how='all'` 옵션을 활용해 보세요.
- ✓ `fillna()`에는 평균(mean), 중앙값(median), 최빈값(mode) 등도 사용할 수 있습니다.
- ✓ `df.describe()`를 통해 대체 전후의 요약 통계를 비교해 보세요!



주의 사항

- 결측치를 무조건 제거하면 데이터가 부족해질 수 있습니다.
- 데이터의 특성에 맞는 대체 방법을 선택해야 합니다.
- 평균/중앙값 대체는 이상치의 영향을 받을 수 있습니다.
- 분석 목적에 따라 적절한 전처리 전략을 선택하세요.

★ 핵심 정리

- ✓ 결측치는 `isna()`로 확인할 수 있습니다.
- ✓ `dropna()`로 제거하거나, `fillna()`로 대체할 수 있습니다.
- ✓ 적절한 결측치 처리는 분석의 정확도와 신뢰도를 높입니다!



교수자 팁

결측치 처리 후, 상관분석(실습 8)이나 그룹분석(실습 9)을 다시 수행하여 결과가 어떻게 달라지는지 비교해 보세요!



데이터 저장과 불러오기 (I/O)

: CSV 파일로 저장하고, 다시 불러와서 확인하기



실습 목표

DataFrame을 CSV 파일로 저장하고,
다시 불러와 데이터가 동일하게 유지되는지 확인합니다.



핵심 개념

- `to_csv()` : DataFrame을 CSV 파일로 저장
- `read_csv()` : CSV 파일을 읽어 DataFrame으로 변환
- `index=False` : 행 인덱스를 파일에 저장하지 않음(권장)
- 파일 경로 : 현재 폴더 기준 상대경로 또는 절대경로 사용



우리 데이터: student_master.csv

	이름	성별	학년	학과	지역	동아리	국어	영어	수학	Python	데이터분석	진로희망	...
0	김민수	남	2	컴퓨터공학	서울	축구	95	85	88	92	90	개발자	...
1	이서연	여	1	데이터사이언스	경기	독서	92	90	92	96	94	데이터분석가	...
2	박지훈	남	3	AI융합	인천	코딩	88	78	82	88	86	AI엔지니어	...
3	최유나	여	2	인공지능	부산	농구	97	93	94	97	95	AI엔지니어	...
4	정현우	남	1	컴퓨터공학	대전	코딩	90	84	87	83	89	개발자	...

... 50 rows × 14 columns

★ 이번 실습에서 사용할 열 : 출력 시 전체 컬럼(14개) 사용

1 CSV 파일 저장

```
# CSV 파일로 저장 (인덱스 제외)
df.to_csv('student_master_out.csv',
          index=False, encoding='utf-8-sig')
print('파일이 저장되었습니다!')
```

✓ student_master_out.csv 파일 생성 완료!

현재 폴더 (예시)

student_master.csv

student_master_out.csv (생성됨)

★ `index=False` 로 인덱스(0,1,2,...)는 저장하지 않음

2 CSV 파일 내용 확인 (일부)

```
# 저장된 CSV 파일 일부 내용 확인
!head -n 6 student_master_out.csv
```

이름,성별,학년,학과,지역,동아리,출석률,국어,영어,수학,Python,데이터분석,진로희망

김민수,남,2,컴퓨터공학,서울,축구,95,85,88,90,92,90,개발자

이서연,여,1,데이터사이언스,경기,독서,92,90,92,89,96,94,데이터분석가

박지훈,남,3,AI융합,인천,코딩,88,78,82,85,88,86,AI엔지니어

최유나,여,2,인공지능,부산,농구,97,93,95,94,97,95,AI엔지니어

정현우,남,1,컴퓨터공학,대전,코딩,90,84,87,83,90,89,개발자

★ CSV 파일은 싼표(,)로 구분된 텍스트 파일입니다.

3 CSV 파일 불러오기

```
# CSV 파일을 다시 읽어오기
df2 = pd.read_csv('student_master_out.csv')
print('불러온 데이터 크기:', df2.shape)
df2.head(3)
```

	이름	성별	학년	학과	지역	동아리	...
0	김민수	남	2	컴퓨터공학	서울	축구	...
1	이서연	여	1	데이터사이언스	경기	독서	...
2	박지훈	남	3	AI융합	인천	코딩	...

불러온 데이터 크기: (50, 14)

★ DataFrame으로 정상적으로 불러옴!

4 원본 데이터와 비교

```
# 원본(df)과 불러온 데이터(df2) 비교
df.equals(df2)
```

✓ True

equals()란?

두 DataFrame의 값과 인덱스, 컬럼이 모두 동일하면 True를 반환합니다.

★ True → 데이터가 동일하게 유지됨!

5 불러온 데이터 기본 정보

df2.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 14 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   이름        50 non-null    object
1   성별        50 non-null    object
2   학년        50 non-null    int64
3   학과        50 non-null    object
4   지역        50 non-null    object
... (총 14개 컬럼) ...
dtypes: int64(5), float64(6), object(3)
memory usage: 5.6+ KB
```

★ 결측치 없이 50행 × 14열 정상 로드!



실습 팁

- ✓ CSV 저장 시 인덱스는 제외하는 것이 일반적입니다 (`index=False`).
- ✓ 한글이 깨질 경우 `encoding='utf-8-sig'` 를 사용하세요.
- ✓ 파일 경로를 지정하고 싶다면 다음과 같이 사용합니다.
예) `df.to_csv('data/student_master_out.csv', index=False)`
- ✓ `read_csv()` 의 주요 옵션: `sep=',', encoding='utf-8', header=0`



주의 사항

- 저장 위치에 쓰기 권한이 없으면 파일이 저장되지 않습니다.
- 파일이 열려 있으면 저장이 안 될 수 있으니 닫은 후 실행하세요.
- 불러올 때 컬럼명이 다르면 `equals()` 결과가 False가 됩니다.

★ 핵심 정리

- ✓ DataFrame을 CSV로 저장하고 불러오는 기본 I/O를 실습했습니다.
- ✓ `to_csv()` 와 `read_csv()` 는 데이터 분석의 가장 기본이 되는 기능입니다.
- ✓ 데이터를 안전하게 저장하고, 재사용할 수 있습니다!



교수자 팁

학생이 다양한 파일 경로(상대경로/절대경로)를 시도해 보게 하고, 다른 파일명으로 저장 후 다시 불러오는 연습을 확장해 보세요!

실습 12

Pandas와 DataFrame 시작하기



우리 데이터: student_master.csv

데이터 필터링과 조건 조회 (Filtering)

: 조건을 사용하여 필요한 데이터만 추출하고 조회하기



실습 목표

조건식을 사용하여 원하는 데이터를 필터링하고, 다양한 방법으로 조회·분석하는 방법을 익힙니다.



핵심 개념

- 조건식 : 비교 연산자(>, <, >=, <=, ==, !=)와 논리 연산자(&, |, ~) 사용
- 불리언 인덱싱 : 조건식 결과(True/False)를 이용해 행 추출
- 여러 조건 결합 : & (AND), | (OR), ~ (NOT)
- 메서드 활용 : query(), isin(), between() 등

	이름	성별	학년	학과	지역	동아리	출석률	국어	영어	수학	Python	데이터분석	진로희망	...
0	김민수	남	2	컴퓨터공학	서울	축구	95	85	88	90	92	90	개발자	...
1	이서연	여	1	데이터사이언스	경기	독서	92	90	92	89	96	94	데이터분석가	...
2	박지훈	남	3	AI융합	인천	코딩	88	78	82	85	88	86	AI엔지니어	...
3	최유나	여	2	인공지능	부산	농구	97	93	95	94	97	95	AI엔지니어	...
4	정현우	남	1	컴퓨터공학	대전	코딩	90	84	87	83	90	89	개발자	...

... 50 rows × 14 columns

★ 이번 실습에서 사용할 열 : 이름, 성별, 학년, 학과, 지역, 동아리, 출석률, 국어, 영어, 수학, Python, 데이터분석, 진로희망

1 단일 조건 필터링

출석률이 90 이상인 학생
df[df['출석률'] >= 90].head()

	이름	출석률	국어	영어	수학
0	김민수	95	85	88	90
1	이서연	92	90	92	89
3	최유나	97	93	95	94
4	정현우	90	84	87	83
5	한지민	94	91	89	92

★ 출석률이 90 이상인 학생만 추출

2 다중 조건 필터링 (AND)

출석률이 90 이상이고, Python 점수가 90 이상
df[(df['출석률'] >= 90) & (df['Python'] >= 90)].head()

	이름	출석률	Python	학과
0	김민수	95	92	컴퓨터공학
1	이서연	92	96	데이터사이언스
3	최유나	97	97	인공지능

★ 두 조건을 모두 만족하는 학생

3 다중 조건 필터링 (OR)

학과가 'AI융합' 또는 '인공지능'
df[(df['학과'] == 'AI융합') | (df['학과'] == '인공지능')].head()

	이름	학과	국어	Python
2	박지훈	AI융합	78	88
3	최유나	인공지능	93	97
8	이준호	AI융합	85	91
11	오수빈	AI융합	92	96
14	홍세영	데이터사이언스	79	89

★ 두 학과 중 하나에 해당하는 학생

4 문자열 포함 필터링

지역이 '서울'인 학생
df[df['지역'] == '서울'].head()

	이름	지역	학과	출석률
0	김민수	서울	컴퓨터공학	95
6	전하운	서울	AI융합	94
10	윤서현	서울	컴퓨터공학	93
16	김태희	서울	AI융합	92

★ 특정 값을 포함하는 행 추출

5 isin() 활용

축구 또는 코딩 동아리 학생
df[df['동아리'].isin(['축구', '코딩'])].head()

	이름	동아리	출석률	Python
0	김민수	축구	95	92
2	박지훈	코딩	88	88
4	정현우	코딩	90	90
7	강민지	축구	92	91
9	이수현	코딩	91	89

★ 여러 값을 한 번에 조건으로 지정

6 query() 메서드 활용

국어 80 이상이고, 수학 85 이상
df.query('국어 >= 80 and 수학 >= 85').head()

	이름	국어	수학	학과
0	김민수	85	90	컴퓨터공학
1	이서연	90	89	데이터사이언스
3	최유나	93	94	인공지능
4	정현우	84	87	컴퓨터공학
5	한지민	91	89	AI융합

★ query()로 가독성 높게 필터링



실습 팁

- ✓ 조건식의 괄호 ()를 정확히 사용하세요.
- ✓ 문자열 비교 시 ==, 대소문자 구분에 주의하세요.
- ✓ 여러 조건을 결합할 때는 & (AND), | (OR)를 사용하고 각 조건을 괄호로 묶어 우선순위를 명확히 합니다.
- ✓ 결과가 너무 많을 때는 .head(), .tail()로 일부만 확인하세요.



주의 사항

- 조건식에서 NaN 값이 포함되면 결과가 예상과 다를 수 있습니다.
→ 필요하면 결측치를 먼저 처리하세요.
- 문자열 비교는 공백이나 오타가 없는지 확인하세요.
- 논리 연산자 사용 시 괄호 누락으로 오류가 발생할 수 있습니다.

★ 핵심 정리

- ✓ 조건식을 이용하면 원하는 데이터만 선택적으로 조회할 수 있습니다.
- ✓ 불리언 인덱싱, isin(), query() 등으로 다양한 필터링이 가능합니다.
- ✓ 필터링은 데이터 분석의 출발점이며, 이후 통계·시각화 등으로 연결되는 핵심 단계입니다.



교수자 팁

다양한 조건을 조합하여 필터링해 보고, 필요한 데이터를 정확히 추출하는 연습을 반복해 보세요. 실무에서는 올바른 데이터 선택이 분석의 품질을 좌우합니다!



학습 목표

여러 조건을 조합하여 원하는 데이터를 추출하는 방법을 익혀봅니다.
student_master.csv 파일을 활용하여 아래 문제를 풀어보세요.

문제 1 출석률 90 이상 AND Python 점수 90 이상



힌트

(조건1) & (조건2) 를 사용하세요.

코드 입력

???

▶ 예상 결과 (일부)

이름	학과	성별	출석률	Python	...
박지훈	AI융합	남	95	95	...
이수민	데이터사이언스	여	96	92	...
최유나	인공지능	여	92	93	...
...	...	...	...	...	...

▶ 출력 예시 : `df[(df['출석률'] >= 90) & (df['Python'] >= 90)]`

문제 2 AI융합 학과 OR 데이터사이언스 학과



힌트

(조건1) | (조건2) 를 사용하세요.

코드 입력

???

▶ 예상 결과 (일부)

이름	학과	학년	성별	...
김민수	컴퓨터공학	2	남	...
박지훈	AI융합	3	남	...
이서연	데이터사이언스	1	여	...
...	...	...	...	...

▶ 출력 예시 : `df[(df['출석률'] == 'AI융합') | (df['학과'] == '데이터사이언스')]`

문제 3 여학생 중 Python 점수 85 이상



힌트

(조건1) & (조건2) 를 사용하세요.

코드 입력

???

▶ 예상 결과 (일부)

이름	학과	성별	Python	...
이서연	데이터사이언스	여	88	...
최유나	인공지능	여	88	...
한지민	AI융합	여	89	...
...	...	...	...	...

▶ 출력 예시 : `df[(df['성별'] == '여') & (df['Python'] >= 85)]`

★ 조건식 조합 방법

- AND 조건 : (조건1) & (조건2)
- OR 조건 : (조건1) | (조건2)
- 괄호 () 로 조건의 우선순위를 지정합니다.

★ 비교 연산자 예시

연산자	의미
==	같다
!=	같지 않다
>	크다
>=	크거나 같다
<	작다
<=	작거나 같다

★ 현재 데이터 미리보기 (일부)

이름	학과	성별	출석률	Python	...
김민수	컴퓨터공학	남	85	85	...
이서연	데이터사이언스	여	92	88	...
박지훈	AI융합	남	95	95	...
...	...	...	...	...	...



실습 시간 : 7분

문제를 풀고 결과를 확인해 보세요!



혼자서 해결해 보세요!

각 문제를 스스로 해결한 후
결과를 확인하고 비교해 보세요.



체크 포인트

- ✓ AND(&), OR(|) 연산자를 정확히 사용할 수 있다.
- ✓ 여러 조건을 조합하여 원하는 데이터를 추출할 수 있다.
- ✓ 괄호를 사용하여 조건의 우선순위를 지정할 수 있다.





데이터 정렬하기 (Sorting)

: 기준 열을 선택하여 오름차순/내림차순으로 데이터를 정렬하기



실습 목표

특정 열을 기준으로 데이터를 오름차순(ascending) 또는 내림차순(descending)으로 정렬하고, 여러 열을 기준으로 정렬해 봅니다.



핵심 개념

- `sort_values()` : 데이터를 정렬 (by, ascending, inplace 등)
- 오름차순 : `ascending=True` (기본값) / 내림차순 : `ascending=False`
- 여러 열 정렬 : `by=['열1', '열2']` (앞의 열을 우선 기준으로 정렬)
- 정렬은 원본을 변경하지 않고 새로운 DataFrame을 반환합니다.

	이름	성별	학년	학과	지역	동아리	출석률	국어	영어	수학	Python	데이터분석	진로희망	...
0	김민수	남	2	컴퓨터공학	서울	축구	95	85	88	90	92	90	개발자	...
1	이서연	여	1	데이터사이언스	경기	독서	92	90	92	89	96	94	데이터분석가	...
2	박지훈	남	3	AI융합	인천	코딩	88	78	82	85	88	86	AI엔지니어	...
3	최유나	여	2	인공지능	부산	농구	97	93	95	94	97	95	AI엔지니어	...
4	정현우	남	1	컴퓨터공학	대전	코딩	90	84	87	83	90	89	개발자	...
...	50 rows x 14 columns													

★ 이번 실습에서 사용할 열 : 이름, 학년, 출석률, 국어, 영어, 수학, Python

1 출석률 기준 오름차순 정렬 (기본)

```
# 출석률 기준 오름차순 정렬 (기본: ascending=True)
df_sort1 = df.sort_values(by='출석률')
df_sort1[['이름', '출석률']].head(5)
```

	이름	출석률
7	강민지	78
12	조유진	80
2	박지훈	88
4	정현우	90
1	이서연	92

낮은 값
(78)
↓
높은 값
(92)

★ 출석률이 낮은 학생부터 높은 학생 순으로 정렬

2 출석률 기준 내림차순 정렬

```
# 출석률 기준 내림차순 정렬
df_sort2 = df.sort_values(by='출석률',
                           ascending=False)
df_sort2[['이름', '출석률']].head(5)
```

	이름	출석률
3	최유나	97
0	김민수	95
1	이서연	92
4	정현우	90
2	박지훈	88

높은 값
(97)
↓
낮은 값
(88)

★ 출석률이 높은 학생부터 낮은 학생 순으로 정렬

3 국어 점수 기준 내림차순 정렬

```
# 국어 점수 기준 내림차순 정렬
df_sort3 = df.sort_values(by='국어',
                           ascending=False)
df_sort3[['이름', '국어']].head(5)
```

	이름	국어
3	최유나	97
0	김민수	95
1	이서연	92
2	박지훈	91
5	한지민	90

높은 점수
(97)
↓
낮은 점수
(90)

★ 국어 점수가 높은 학생부터 정렬

4 여러 열 기준 정렬 (학년 ↑, 출석률 ↓)

```
# 1순위: 학년 오름차순, 2순위: 출석률 내림차순
df_sort4 = df.sort_values(
    by=['학년', '출석률'],
    ascending=[True, False]
)
df_sort4[['학년', '이름', '출석률']].head(6)
```

학년	이름	출석률
1	이서연	92
1	정현우	90
1	한지민	94
2	최유나	97
2	김민수	95
2	강민지	78

정렬 우선순위
① 학년
(낮은 값 우선)
② 출석률
(높은 값 우선)

★ 학년이 낮은 순으로, 같은 학년은 출석률 높은 순으로 정렬



실습 팁

- ✓ `ascending=False` 로 내림차순 정렬을 손쉽게 할 수 있습니다.
- ✓ 여러 열을 지정하면 앞의 열을 기준으로 먼저 정렬하고, 같은 값끼리는 다음 열 기준으로 정렬합니다.
- ✓ `na_position='first'` 또는 `'last'` 로 NaN의 위치를 지정할 수 있습니다.

```
df.sort_values(by='출석률', ascending=False, na_position='last')
```



주의 사항

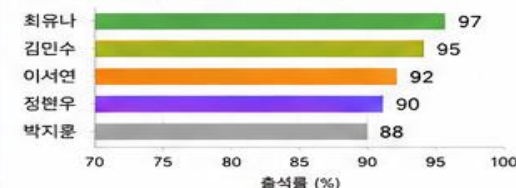
- `sort_values()` 는 원본을 변경하지 않습니다. (정렬 결과는 새로운 DataFrame으로 반환됩니다.)
- 정렬 기준이 되는 열의 데이터 타입이 일관되어야 합니다.
- 문자열 정렬은 사전식(가나다 순)으로 동작합니다.



추가 실습 아이디어

- 수학 점수 기준 오름차순/내림차순 정렬
- 출석률과 Python 점수를 기준으로 복합 정렬
- 특정 학과(예: 'AI융합') 학생들만 필터링 후 정렬
- 정렬 결과를 index 리셋하여 보기
`df_sort4.reset_index(drop=True)`

정렬 결과 시각화 예시 (출석률 기준)





데이터 집계와 피벗 테이블 (Pivot Table)

: 데이터를 다양한 기준으로 집계하고 피벗 테이블로 재구성하기



실습 목표

groupby()와 agg()를 이용해 데이터를 집계하고, pivot_table()을 사용해 피벗 테이블로 재구성합니다.



핵심 개념

- groupby() + agg() : 그룹별 집계 (합계, 평균, 개수 등)
- pivot_table() : 데이터를 행/열 기준으로 피벗(교차)하여 요약
- fill_value : 결측치를 특정 값으로 채우기
- margins=True : 합계(총계) 행/열 추가

우리 데이터: student_master.csv

	이름	성별	학년	학과	지역	동아리	출석률	국어	영어	수학	Python	진로희망	...
0	김민수	남	2	컴퓨터공학	서울	축구	95	85	88	90	90	개발자	...
1	이서연	여	1	데이터사이언스	경기	독서	92	90	92	99	94	데이터분석가	...
2	박지훈	남	3	AI융합	인천	코딩	88	78	82	85	86	AI엔지니어	...
3	최유나	여	2	인공지능	부산	농구	97	93	95	94	97	AI엔지니어	...
4	정현우	남	1	컴퓨터공학	대전	코딩	90	84	87	83	89	개발자	...

... 50 rows x 14 columns

★ 이번 실습에서 사용할 열 : 학년, 학과, 성별, 지역, 출석률, 국어, 영어, 수학, Python, 데이터분석

1 학과별 평균 점수 집계

```
# 학과별 평균 점수(국어, 영어, 수학, Python, 데이터분석)
df_group = df.groupby('학과')[['국어', '영어', '수학', 'Python', '데이터분석']].mean().round(2)
df_group
```

학과	국어	영어	수학	Python	데이터분석
AI융합	90.67	85.33	89.00	88.00	87.33
데이터사이언스	90.50	89.50	91.00	92.00	93.00
컴퓨터공학	86.80	82.20	85.40	87.40	88.60
인공지능	93.50	93.00	95.00	94.50	95.00

★ 학과별 전반적인 학업 성취도를 비교할 수 있어요!

2 성별 × 학년별 평균 출석률

```
# 성별과 학년별 평균 출석률
df_pivot1 = pd.pivot_table(df,
    values='출석률', index='학년', columns='성별',
    aggfunc='mean', round(2))
df_pivot1
```

성별	여	남
학년		
1	91.50	89.67
2	95.00	92.50
3	96.00	90.00

★ 학년이 높아질수록 전체 출석률이 높아져요!

3 학과 × 성별별 Python 평균 점수 (피벗)

```
# 학과와 성별별 Python 평균 점수
df_pivot2 = pd.pivot_table(df,
    values='Python', index='학과', columns='성별',
    aggfunc='mean', round(2), fill_value=0)
df_pivot2
```

성별	여	남
학과		
AI융합	89.00	87.00
데이터사이언스	93.00	91.00
컴퓨터공학	85.50	88.50
인공지능	95.50	94.00

★ 여학생이 전반적으로 Python 점수가 높아요!

4 지역별 학생 수와 평균 출석률 (집계)

```
# 지역별 학생 수와 평균 출석률
df_group2 = df.groupby('지역').agg(
    학생수=('이름', 'count'),
    평균출석률=('출석률', 'mean')
).round(2).reset_index()
df_group2
```

지역	학생수	평균출석률
대전	5	90.40
부산	6	95.20
서울	18	92.56
인천	6	90.17
경기	15	91.87

★ 부산 지역의 출석률이 가장 높아요!

5 피벗 테이블: 학년 × 과목 평균 점수

```
# 학년별 과목 평균 점수 피벗 테이블
df_pivot3 = pd.pivot_table(df,
    values=['국어', '영어', '수학', 'Python', '데이터분석'],
    index='학년', aggfunc='mean', round(2),
    margins=True, margins_name='전체 평균')
df_pivot3
```

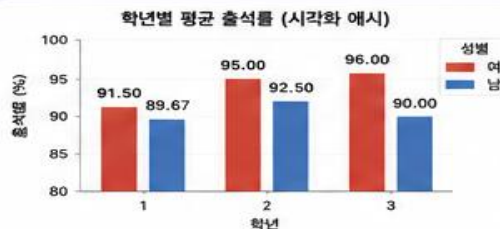
	국어	영어	수학	Python	데이터분석
학년					
1	86.25	84.25	86.75	87.25	88.25
2	91.30	89.40	91.80	92.10	92.80
3	93.80	92.60	94.40	94.20	94.60
전체 평균	90.55	88.85	90.98	91.18	91.88

★ 전체 평균을 통해 학년 간 성취도를 한눈에 비교!



실습 팁

- ✓ groupby() + agg()는 자유롭게 집계 함수를 조합할 수 있어요!
- ✓ pivot_table()의 fill_value는 결측치를 0 또는 다른 값으로 채웁니다.
- ✓ margins=True를 사용하면 전체 합계/평균을 쉽게 확인할 수 있어요!



주의 사항

- 집계 대상 열은 숫자형 데이터여야 합니다.
- pivot_table()에서 index, columns로 사용할 열은 범주형 데이터가 적합합니다.
- 결측치가 있으면 평균 계산에 영향을 줄 수 있으므로 필요시 결측치 처리를 먼저 수행하세요.
- 결과 해석 시 전체 데이터 분포와 함께 고려하세요.

핵심 정리

- ✓ groupby()와 agg()로 데이터를 요약할 수 있습니다.
- ✓ pivot_table()로 교차 분석 테이블을 쉽게 만들 수 있습니다.
- ✓ fill_value와 margins 옵션을 활용하면 다양한 분석이 가능합니다.
- ✓ 집계와 피벗 테이블은 EDA(탐색적 데이터 분석)의 핵심 도구입니다!





학습 목표

여러 기준(행/열)을 기준으로 데이터를 요약하여 피벗 테이블을 만드는 방법을 익혀봅니다.

student_master.csv 파일을 활용하여 아래 문제를 풀어보세요.

문제 1 학년별 평균 Python 점수



힌트

행(index)에 '학년'을 지정하고,
값(values)에 'Python'의 평균을 계산하세요.

코드 입력

???

▶ 예상 결과 (예시)

학년	Python (평균)
1	90.2
2	88.7
3	87.1
4	85.6

▶ 출력 예시 : `df.pivot_table(index='학년',
values='Python', aggfunc='mean')`

문제 2 성별 평균 출석률



힌트

행(index)에 '성별'을 지정하고,
값(values)에 '출석률'의 평균을 계산하세요.

코드 입력

???

▶ 예상 결과 (예시)

성별	출석률 (평균)
남	93.2
여	94.7

▶ 출력 예시 : `df.pivot_table(index='성별',
values='출석률', aggfunc='mean')`

문제 3 학과 x 성별 평균 데이터분석 점수



힌트

행(index)에 '학과', 열(columns)에 '성별'을 지정하고,
값(values)에 '데이터분석'의 평균을 계산하세요.

코드 입력

???

▶ 예상 결과 (예시)

학과	성별	
	남	여
AI융합	86.4	88.7
데이터사이언스	88.9	90.5
컴퓨터공학	87.6	99.1
인공지능	89.3	90.8

▶ 출력 예시 : `df.pivot_table(index='학과', columns='성별',
values='데이터분석', aggfunc='mean')`

★ pivot_table 기본 문법

```
df.pivot_table(
    index=...,      # 행 기준
    columns=...,    # 열 기준 (선택)
    values=...,     # 값
    aggfunc='mean'  # 집계 함수
)
```

※ aggfunc : 'mean', 'sum', 'count',
'min', 'max' 등 사용 가능

★ 자주 사용하는 집계 함수

- mean : 평균
- sum : 합계
- count : 개수
- min : 최솟값
- max : 최댓값

★ 데이터 미리보기 (일부)

이름	학과	학년	성별	출석률	Python	데이터분석	...
김민수	컴퓨터공학	2	남	95	85	78	...
이서연	데이터사이언스	1	여	92	92	88	...
박지훈	AI융합	3	남	95	95	82	...
...	...	...	...	...	...	...	...



실습 시간 : 8분

문제를 풀고 결과를 확인해 보세요!



혼자서 해보세요!

각 문제를 스스로 해결한 후
결과를 확인하고 비교해 보세요.



체크 포인트

- ✓ index(행)와 columns(열)을 올바르게 지정했는가?
- ✓ aggfunc를 사용하여 원하는 집계 결과를 얻었는가?
- ✓ 결과 테이블의 구조가 이해되었는가?





데이터 집계와 피벗 테이블 (Pivot Table)

: 데이터를 다양한 기준으로 집계하고 피벗 테이블로 재구성하기



실습 목표

groupby()와 agg()를 이용해 데이터를 집계하고, pivot_table()을 사용해 피벗 테이블로 재구성합니다.



핵심 개념

- groupby() + agg() : 그룹별 집계 (합계, 평균, 개수 등)
- pivot_table() : 데이터를 행/열 기준으로 피벗(교차)하여 요약
- fill_value : 결측치를 특정 값으로 채우기
- margins=True : 합계(총계) 행/열 추가

우리 데이터: student_master.csv

	이름	성별	학년	학과	지역	동아리	출석률	국어	영어	수학	Python	진로희망	...
0	김민수	남	2	컴퓨터공학	서울	축구	95	85	88	90	90	개발자	...
1	이서연	여	1	데이터사이언스	경기	독서	92	90	92	99	94	데이터분석가	...
2	박지훈	남	3	AI융합	인천	코딩	88	78	82	85	86	AI엔지니어	...
3	최유나	여	2	인공지능	부산	농구	97	93	95	94	97	AI엔지니어	...
4	정현우	남	1	컴퓨터공학	대전	코딩	90	84	87	83	89	개발자	...

... 50 rows x 14 columns

★ 이번 실습에서 사용할 열 : 학년, 학과, 성별, 지역, 출석률, 국어, 영어, 수학, Python, 데이터분석

1 학과별 평균 점수 집계

```
# 학과별 평균 점수(국어, 영어, 수학, Python, 데이터분석)
df_group = df.groupby('학과')[['국어', '영어', '수학', 'Python', '데이터분석']].mean().round(2)
df_group
```

학과	국어	영어	수학	Python	데이터분석
AI융합	90.67	85.33	89.00	88.00	87.33
데이터사이언스	90.50	89.50	91.00	92.00	93.00
컴퓨터공학	86.80	82.20	85.40	87.40	88.60
인공지능	93.50	93.00	95.00	94.50	95.00

★ 학과별 전반적인 학업 성취도를 비교할 수 있어요!

2 성별 × 학년별 평균 출석률

```
# 성별과 학년별 평균 출석률
df_pivot1 = pd.pivot_table(df,
    values='출석률', index='학년', columns='성별',
    aggfunc='mean', round(2))
df_pivot1
```

성별	여	남
학년		
1	91.50	89.67
2	95.00	92.50
3	96.00	90.00

★ 학년이 높아질수록 전체 출석률이 높아져요!

3 학과 × 성별별 Python 평균 점수 (피벗)

```
# 학과와 성별별 Python 평균 점수
df_pivot2 = pd.pivot_table(df,
    values='Python', index='학과', columns='성별',
    aggfunc='mean', round(2), fill_value=0)
df_pivot2
```

성별	여	남
학과		
AI융합	89.00	87.00
데이터사이언스	93.00	91.00
컴퓨터공학	85.50	88.50
인공지능	95.50	94.00

★ 여학생이 전반적으로 Python 점수가 높아요!

4 지역별 학생 수와 평균 출석률 (집계)

```
# 지역별 학생 수와 평균 출석률
df_group2 = df.groupby('지역').agg(
    학생수=('이름', 'count'),
    평균출석률=('출석률', 'mean')
).round(2).reset_index()
df_group2
```

지역	학생수	평균출석률
대전	5	90.40
부산	6	95.20
서울	18	92.56
인천	6	90.17
경기	15	91.87

★ 부산 지역의 출석률이 가장 높아요!

5 피벗 테이블: 학년 × 과목 평균 점수

```
# 학년별 과목 평균 점수 피벗 테이블
df_pivot3 = pd.pivot_table(df,
    values=['국어', '영어', '수학', 'Python', '데이터분석'],
    index='학년', aggfunc='mean', round(2),
    margins=True, margins_name='전체 평균')
df_pivot3
```

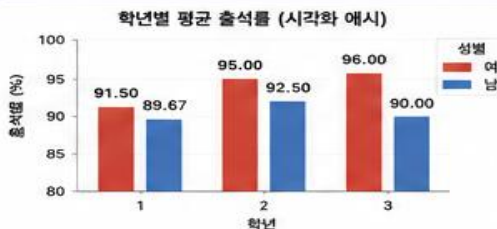
	국어	영어	수학	Python	데이터분석
학년					
1	86.25	84.25	86.75	87.25	88.25
2	91.30	89.40	91.80	92.10	92.80
3	93.80	92.60	94.40	94.20	94.60
전체 평균	90.55	88.85	90.98	91.18	91.88

★ 전체 평균을 통해 학년 간 성취도를 한눈에 비교!



실습 팁

- ✓ groupby() + agg()는 자유롭게 집계 함수를 조합할 수 있어요!
- ✓ pivot_table()의 fill_value는 결측치를 0 또는 다른 값으로 채웁니다.
- ✓ margins=True를 사용하면 전체 합계/평균을 쉽게 확인할 수 있어요!



주의 사항

- 집계 대상 열은 숫자형 데이터여야 합니다.
- pivot_table()에서 index, columns로 사용할 열은 범주형 데이터가 적합합니다.
- 결측치가 있으면 평균 계산에 영향을 줄 수 있으므로 필요시 결측치 처리를 먼저 수행하세요.
- 결과 해석 시 전체 데이터 분포와 함께 고려하세요.

핵심 정리

- ✓ groupby()와 agg()로 데이터를 요약할 수 있습니다.
- ✓ pivot_table()로 교차 분석 테이블을 쉽게 만들 수 있습니다.
- ✓ fill_value와 margins 옵션을 활용하면 다양한 분석이 가능합니다.
- ✓ 집계와 피벗 테이블은 EDA(탐색적 데이터 분석)의 핵심 도구입니다!





학습 목표

여러 가지 데이터를 시각화하여 패턴과 특징을 파악하는 방법을 익혀봅니다.
student_master.csv 파일을 활용하여 아래 문제를 풀어보세요.

문제 1 학년별 평균 출석률 그래프



힌트

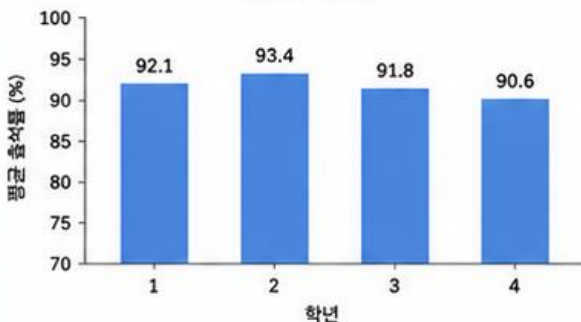
groupby('학년')['출석률'].mean()으로 평균을 구하고
막대 그래프(bar)를 그려보세요.

코드 입력

???

▶ 예상 결과 (예시)

학년별 평균 출석률



▶ 사용 함수/에서드 : groupby(), mean(), plot(kind='bar')

문제 2 성별 학생 수 그래프



힌트

value_counts()로 성별 학생 수를 구하고
파이 차트(pie)를 그려보세요.

코드 입력

???

▶ 예상 결과 (예시)

성별 학생 수



▶ 사용 함수/에서드 : value_counts(), plot(kind='pie')

문제 3 과목별 평균 점수 그래프



힌트

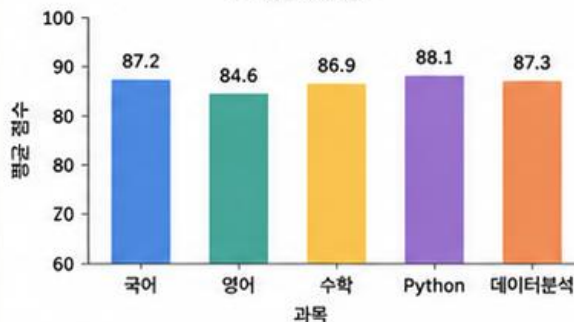
여러 과목의 평균을 구한 후
막대 그래프(bar)를 그려보세요.

코드 입력

???

▶ 예상 결과 (예시)

과목별 평균 점수



▶ 사용 함수/에서드 : mean(), plot(kind='bar')

★ 시각화 기본 팁

- 그래프에는 제목(title)과 축 이름(xlabel, ylabel)을 추가하면 더 보기 좋습니다.
- 색상(color), 크기(figsize)를 변경해 보세요.
- plt.tight_layout()을 사용하면 레이아웃이 깔끔해집니다.

★ 주요 시각화 함수

함수/에서드	설명	예시
plot(kind='bar')	막대 그래프	df.plot(kind='bar')
plot(kind='pie')	파이 차트	s.plot(kind='pie')
plot(kind='line')	선 그래프	df.plot(kind='line')
hist()	히스토그램	df['컬럼'].hist()

★ 현재 데이터 미리보기 (일부)

이름	학년	성별	출석률	국어	Python	데이터분석	...
김민수	2	남	95	85	85	78	...
이서연	1	여	96	88	92	88	...
박지훈	3	남	95	95	95	82	...
...	...	...	...	...	...	...	...



주의!

한글 폰트가 깨지면 plt.rcParams['font_family']
설정으로 한글 폰트를 지정하세요.



실습 시간 : 8분

문제를 풀고 결과를 확인해 보세요!



혼자서 해결해 보세요!

각 문제를 스스로 해결한 후
결과를 확인하고 비교해 보세요.



체크 포인트

- ✓ 데이터를 적절히 집계하여 시각화할 수 있다.
- ✓ 막대 그래프와 파이 차트를 만들 수 있다.
- ✓ 제목, 축 이름, 레이아웃을 설정하여 완성도 높은 그래프를 만들 수 있다.





데이터 시각화 (Visualization)

: pandas와 matplotlib을 이용해 데이터를 시각화해 보자



실습 목표

pandas의 plot()과 matplotlib을 이용하여 다양한 차트로 데이터를 시각화하고 해석하는 방법을 익힙니다.



핵심 개념

- plot() : pandas에서 제공하는 간편한 시각화 기능
- matplotlib : 더 세밀하고 다양한 시각화 제공
- kind 종류 : 'bar', 'line', 'pie', 'hist', 'scatter' 등
- 레이블, 제목, 범례, 색상, 스타일 설정

	이름	성별	학년	학과	지역	동아리	출석률	국어	영어	수학	Python	데이터분석	...
0	김민수	남	2	컴퓨터공학	서울	축구	95	85	88	90	90	개발자	...
1	이서연	여	1	데이터사이언스	경기	독서	92	90	92	96	94	데이터분석가	...
2	박지훈	남	3	AI융합	인천	코딩	88	78	82	88	86	AI엔지니어	...
3	최유나	여	2	인공지능	부산	농구	97	93	95	97	95	AI엔지니어	...
4	정현우	남	1	컴퓨터공학	대전	코딩	90	84	87	83	89	개발자	...
...	50 rows x 14 columns												

★ 이번 실습에서 사용할 열 : 성별, 학년, 출석률, 국어, 영어, 수학, Python, 데이터분석, 진로희망

1 성별 학생 수 (막대그래프)

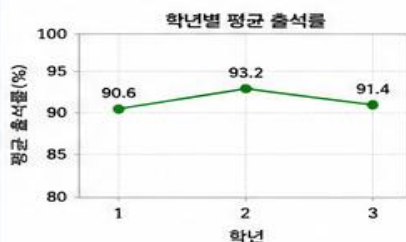
```
# 성별 학생 수 시각화
df['성별'].value_counts().plot(
    kind='bar', color=['skyblue', 'pink']
)
plt.title('성별 학생 수')
plt.xlabel('성별')
plt.ylabel('학생 수')
plt.grid(axis='y', linestyle='--', alpha=0.5)
```



★ 남학생이 더 많아요!

2 학년별 평균 출석률 (선그래프)

```
# 학년별 평균 출석률 시각화
df.groupby('학년')['출석률'].mean().plot(
    kind='line', marker='o', color='green'
)
plt.title('학년별 평균 출석률')
plt.xlabel('학년')
plt.ylabel('평균 출석률(%)')
plt.ylim(80, 100)
plt.grid(True, linestyle='--', alpha=0.5)
```



★ 2학년의 출석률이 가장 높아요!

3 과목별 평균 점수 (막대그래프)

```
# 과목별 평균 점수 시각화
subjects = ['국어', '영어', '수학', 'Python', '데이터분석']
df[subjects].mean().plot(kind='bar', color='cornflowerblue')
plt.title('과목별 평균 점수')
plt.ylabel('평균 점수')
plt.ylim(70, 100)
plt.grid(axis='y', linestyle='--', alpha=0.5)
```



★ 데이터분석 점수가 가장 높아요!

4 진로희망 분포 (파이차트)

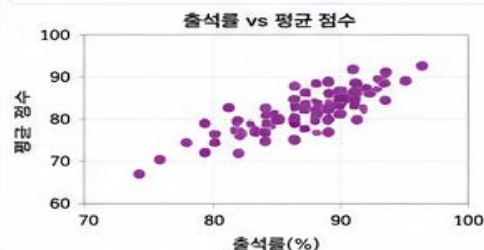
```
# 진로희망 분포 시각화
df['진로희망'].value_counts().nlargest(5).plot(
    kind='pie', autopct='%1.1f%%', startangle=90,
    colors=plt.cm.Set3.colors
)
plt.title('진로희망 분포 (상위 5개)')
plt.ylabel('')
```



★ 개발자를 희망하는 학생이 가장 많아요!

5 출석률 vs 평균 점수 (산점도)

```
# 출석률과 평균 점수의 관계 시각화
df['평균점수'] = df[['국어', '영어', '수학', 'Python', '데이터분석']].mean(axis=1)
df.plot(kind='scatter', x='출석률', y='평균점수',
        c='purple', alpha=0.7)
plt.title('출석률 vs 평균 점수')
plt.xlabel('출석률(%)')
plt.ylabel('평균 점수')
plt.grid(True, linestyle='--', alpha=0.5)
```



★ 출석률이 높을수록 성적도 높아요!



실습 팁

- plt.figure(figsize=(8,5)) 로 그림 크기 조절
- plt.title(), plt.xlabel(), plt.ylabel() 로 제목/축 레이블 추가
- plt.grid(True) 로 가독성 향상
- color, marker, linestyle 등을 활용해 스타일을 꾸며보세요!
- df.plot() 은 간단하고, matplotlib은 세밀한 커스터마이징에 유리해요!

주의 사항

- 한글이 깨질 경우, 한글 폰트를 설정하세요.
예) import matplotlib.pyplot as plt
plt.rc('font', family='Malgun Gothic')
- pandas plot은 간단한 차트에 적합하고, 더 복잡한 차트는 matplotlib을 직접 사용하는 것이 좋아요.

핵심 정리

- pandas의 plot()으로 빠르게 시각화할 수 있습니다.
- matplotlib로 제목, 색상, 범례 등 세부 설정이 가능합니다.
- 데이터를 다양한 각도에서 시각화하면 인사이트를 얻기 쉬워요!
- 시각화는 데이터 분석의 결과를 효과적으로 전달하는 도구입니다.

</> 유용한 코드 모음

```
df.plot(kind='line') # 선그래프
s.plot(kind='bar') # 막대그래프
s.plot(kind='pie', autopct='%1.1f%%') # 파이차트
df.plot(kind='scatter', x='A', y='B') # 산점도
plt.show() # 그래프 출력
```



실습 17

Pandas와 DataFrame 시작하기



데이터 분석 종합 실습 2 (Mini Project 2)

: 학생 성적 데이터로부터 통계 분석과 시각화 대시보드 만들기



실습 목표

학생 성적 데이터를 이용하여 과목별/학과별 통계 분석을 수행하고, 시각화 대시보드를 구성하여 핵심 인사이트를 도출합니다.



핵심 개념

- 집계(agg), 그룹화(groupby), 정렬(sort_values), 피벗(pivot_table)
- 다양한 시각화(막대, 선, 히트맵, 파이)로 데이터 인사이트 도출
- 여러 분석 결과를 하나의 대시보드 형태로 구성

우리 데이터: student_scores.csv

	이름	학년	학과	국어	영어	수학	과학	프로그래밍	총점	평균	등급
0	김민수	2	컴퓨터공학	85	90	88	92	95	450	90.0	A
1	이서연	1	데이터사이언스	92	88	90	85	91	446	89.2	A
2	박지훈	3	AI융합	78	82	85	88	80	413	82.6	B+
3	최유나	2	인공지능	95	93	94	97	96	475	95.0	A+
4	정현우	1	컴퓨터공학	89	84	87	83	90	433	86.6	A

... 100 rows x 11 columns

★ 이번 실습에서 사용할 열 : 이름, 학년, 학과, 국어, 영어, 수학, 과학, 프로그래밍, 총점, 평균, 등급

1 데이터 로드 및 기본 확인

```
df = pd.read_csv('student_scores.csv')
df.head()
```

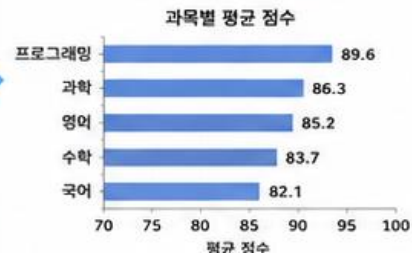
	이름	학년	학과	국어	영어	수학	과학	프로그래밍	총점	평균	등급
0	김민수	2	컴퓨터공학	85	90	88	92	95	450	90.0	A
1	이서연	1	데이터사이언스	92	88	90	85	91	446	89.2	A
2	박지훈	3	AI융합	78	82	85	88	80	413	82.6	B+
3	최유나	2	인공지능	95	93	94	97	96	475	95.0	A+
4	정현우	1	컴퓨터공학	89	84	87	83	90	433	86.6	A

(100, 11)

★ 데이터 형태와 기본 정보를 확인합니다.

2 과목별 평균 점수 비교

```
sub_cols = ['국어', '영어', '수학', '과학', '프로그래밍']
sub_mean = df[sub_cols].mean()
sub_mean.sort_values(ascending=False)
```



★ 프로그래밍 과목의 평균이 가장 높아요!

3 학과별 평균 점수 및 학생 수

```
dept_stats = df.groupby('학과').agg(
    평균점수=('평균', 'mean'),
    학생수=('이름', 'count')
).sort_values('평균점수', ascending=False)
```

학과	평균점수	학생수
AI융합	89.7	22
인공지능	88.3	25
컴퓨터공학	86.9	28
데이터사이언스	85.4	25

★ AI융합 학과의 평균 점수가 가장 높아요!

4 학년별 평균 점수 추세

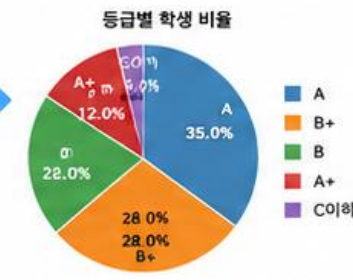
```
grade_mean = df.groupby('학년')['평균']
grade_mean.mean().sort_index()
```



★ 2학년의 평균 점수가 가장 높아요!

5 등급별 학생 비율

```
grade_cnt = df['등급'].value_counts()
grade_pct = grade_cnt / len(df) * 100
```



★ A 등급 비율이 가장 높아요!

6 과목 × 학과 평균 점수 (히트맵)

```
pivot = pd.pivot_table(df,
    values=sub_cols, index='학과',
    aggfunc='mean').round(1)
```

학과	국어	영어	수학	과학	프로그래밍
컴퓨터공학	81.5	84.7	83.2	86.5	88.9
데이터사이언스	81.2	83.1	82.0	84.3	87.0
AI융합	84.3	86.6	84.9	88.6	90.3
인공지능	82.6	85.0	84.6	87.6	89.1

★ AI융합 학과의 점수가 전반적으로 높아요!



실습 팁

- ✓ 결측치가 있다면 df.isnull().sum()으로 확인하고 처리하세요.
- ✓ 수치형 데이터는 round() 로 소수점 자릿수를 조정하세요.
- ✓ 데이터를 다양한 기준으로 비교하고 해석하는 연습을 하세요.
- ✓ 시각화는 제목, 축 라벨, 범례를 명확히 설정하세요.



주의 사항

- 평균 계산 시 NaN 값은 자동으로 제외됩니다.
- sort_values()의 ascending 옵션을 확인하세요.
- pivot_table()은 인덱스와 열을 바꿔가며 활용해 보세요.
- 시각화 시 x축/y축 범위와 눈금 간격을 적절히 설정하세요.

★ 핵심 인사이트 예시

- ✓ 프로그래밍 과목의 평균이 가장 높고, 국어가 가장 낮습니다.
- ✓ AI융합 학과가 전반적으로 우수한 성적을 보입니다.
- ✓ 2학년의 평균 성적이 가장 우수합니다.
- ✓ A 등급 비율이 가장 높아 전체 성적 수준이 양호합니다.

✓ 다음 단계 아이디어

- 학생별 성적 변화 추이 (선그래프)
- 상위/하위 학생 성적 상세 분석
- 과목 난이도 분석 (분산, 표준편차)
- 성적 예측 모델링 (회귀분석)



실제 분석 프로젝트는 '질문 정의 → 데이터 이해 → 전처리/필터링 → 탐색적 분석 → 시각화 → 인사이트 도출 → 보고서 작성'의 흐름으로 진행됩니다.

실습 18

Pandas와 DataFrame 시작하기



데이터 분석 종합 실습 3 (Mini Project 3)

: 데이터를 기반으로 통계 분석, 시각화, 인사이트 도출 및 간단한 예측 모델 만들기



실습 목표

학생 성적 데이터를 활용하여 탐색적 데이터 분석(EDA)을 수행하고, 성적을 예측하는 선형 회귀 모델을 만들어 성능을 평가합니다.



핵심 개념

- EDA : 기술 통계, 상관관계 분석, 그룹별 비교
- 시각화 : 분포, 상관관계, 박스플롯, 히트맵
- 머신러닝 기초 : 선형 회귀 모델, 성능 평가 (MAE, RMSE, R²)



우리 데이터: student_scores.csv

	이름	학년	학과	국어	영어	수학	과학	프로그래밍	총점	평균	등급
0	김민수	2	컴퓨터공학	85	90	88	92	95	450	90.0	A
1	이서연	1	데이터사이언스	92	88	90	85	91	446	89.2	A
2	박지훈	3	AI융합	78	82	85	88	80	413	82.6	B+
3	최유나	2	인공지능	95	93	94	97	96	475	95.0	A+
4	정현우	1	컴퓨터공학	89	84	87	83	90	433	86.6	A

... 100 rows x 11 columns

★ 이번 실습에서 사용할 열 : 이름, 학년, 학과, 국어, 영어, 수학, 과학, 프로그래밍, 총점, 평균, 등급

1 데이터 로드 및 기본 확인

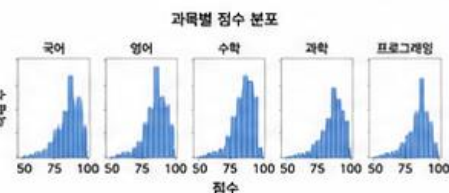
```
import pandas as pd
df = pd.read_csv('student_scores.csv')
df.head()
df.info()
df.describe()
```

	국어	영어	수학	과학	프로그래밍
count	100.0	100.0	100.0	100.0	100.0
mean	84.6	84.9	85.1	85.7	87.3
std	10.2	9.8	9.7	9.9	8.6
min	55.0	58.0	60.0	59.0	62.0
max	100.0	100.0	100.0	100.0	100.0

★ 데이터 구조와 기본 통계치를 확인합니다.

2 성적 분포 시각화

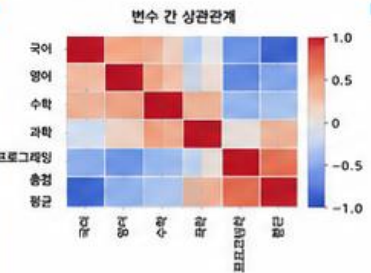
```
import matplotlib.pyplot as plt
import seaborn as sns
subjects = ['국어', '영어', '수학', '과학', '프로그래밍']
df[subjects].hist(bins=15, figsize=(10,3))
plt.suptitle('과목별 점수 분포')
plt.tight_layout()
plt.show()
```



★ 과목별 점수 분포를 파악합니다.

3 상관관계 분석 (히트맵)

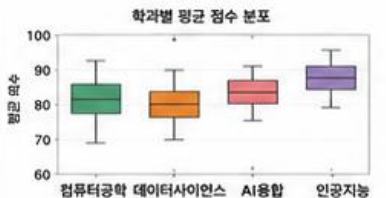
```
corr = df[subjects + ['총점', '평균']].corr()
sns.heatmap(corr, annot=True, cmap='coolwarm',
            vmin=-1, vmax=1, center=0)
plt.title('변수 간 상관관계')
plt.ylabel('과목별 점수')
plt.show()
```



★ 총점/평균과 상관관계가 높은 과목을 확인합니다.

4 학과별 평균 점수 비교 (박스플롯)

```
plt.figure(figsize=(8,4))
sns.boxplot(data=df, x='학과', y='평균', palette='Set2')
plt.title('학과별 평균 점수 분포')
plt.xlabel('학과')
plt.ylabel('평균 점수')
plt.grid(axis='y', linestyle='--', alpha=0.5)
plt.show()
```



★ 학과별 성적 분포 및 중앙 경향을 비교합니다.

5 성적 예측 모델 (선형 회귀)

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, \
    mean_squared_error, r2_score

X = df[subjects]
y = df['총점']

X_train, X_test, y_train, y_test = \
    train_test_split(X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

★ 과목 점수로 총점을 예측하는 모델을 학습합니다.

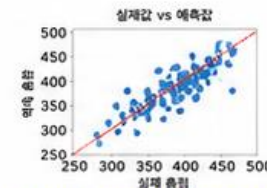
6 모델 성능 평가 및 예측 결과 시각화

```
mae = mean_absolute_error(y_test, y_pred)
rmse = mean_squared_error(y_test, y_pred, squared=False)
r2 = r2_score(y_test, y_pred)

print(f'MAE : {mae:.2f}')
print(f'RMSE : {rmse:.2f}')
print(f'R^2 : {r2:.3f}')

# 실제값 vs 예측값 선형도
plt.figure(figsize=(5,4))
sns.scatterplot(x=y_test, y=y_pred)
plt.plot([y.min(), y.max()], [y.min(), y.max()],
        'r--', label='일대일')
plt.xlabel('실제 총점')
plt.ylabel('예측 총점')
plt.title('실제값 vs 예측값')
plt.legend(); plt.grid(True); plt.show()
```

성능 지표 예시
MAE : 18.35
RMSE : 22.14
R² : 0.842



★ 모델 성능을 평가하고 예측 성능을 시각적으로 확인합니다.



실습 팁

- ✓ 결측치가 있다면 df.isnull().sum()으로 확인 후 처리하세요.
- ✓ 수치형 변수는 describe(), 시각화로 분포를 확인하세요.
- ✓ 상관관계 분석으로 중요한 변수를 파악하세요.
- ✓ 모델 예측 전 데이터를 학습/테스트로 분리하는 것이 중요합니다.



주의 사항

- 데이터 누수를 방지하기 위해 전처리는 학습 데이터에만 적용하세요.
- 회귀 성능 평가는 MAE, RMSE, R²를 함께 해석하세요.
- 과적합이 의심되면 특성 선택 또는 규제(Regularization)를 고려하세요.

핵심 인사이트 예시

- ✓ 프로그래밍, 수학 점수가 총점과 가장 높은 상관관계를 보입니다.
- ✓ 인공지능 학과 학생들의 평균 점수가 가장 높습니다.
- ✓ 선형 회귀 모델의 R²가 0.84로, 예측 성능이 우수합니다.

다음 단계 아이디어

- 다른 알고리즘(랜덤포레스트, XGBoost)로 예측 성능 비교
- 과목 가중치를 반영한 커스텀 총점 예측
- 성적 향상을 위한 학생 맞춤 분석 대시보드 제작



교수자 팁

실제 데이터 분석 프로젝트는 '문제 정의 → 데이터 이해(EDA) → 전처리 → 분석/시각화 → 모델링 → 평가 → 인사이트 도출 → 의사결정 제안'의 흐름으로 진행됩니다.

최종 미션 나만의 성적 분석 리포트 만들기



미션 목표

지금까지 배운 Pandas 기능을 활용하여 학생 성적 데이터를 분석하고, 나만의 인사이트를 담은 분석 리포트를 완성해 보세요!

요구 사항

- student_master.csv 파일을 활용할 것
- 최소 5가지 이상의 Pandas 기능을 사용할 것
- 다양한 시각화 그래프를 포함할 것
- 분석 결과에 대한 해석(인사이트)을 작성할 것

제출물

- ✓ Jupyter Notebook 파일 (.ipynb)
- ✓ 분석 리포트 (PDF 또는 PPT, 2~3페이지)
- ✓ 파일명 : 학번_이름_final.ipynb (예: 20241234_홍길동_final.ipynb)

활용해야 할 기능 (예시)

1 데이터 확인 및 기초 통계

head(), info(), describe() 등



2 열/행 선택 및 필터링

[], loc[], iloc[], 조건 필터링



3 정렬 및 그룹별 분석

sort_values(), groupby()



4 집계 및 통계

mean(), sum(), count(), min(), max()



5 피벗 테이블

pivot_table()



6 시각화

bar, pie, line, hist 등



★ 분석 주제 예시 (다음 중 하나를 선택하거나 자유롭게 주제를 설정하세요)

1 학과별 성적 분석

학과별 평균 점수 비교,
과목별 강/약점 분석

2 성별 성적 비교

성별 과목별 평균 비교,
상위권 비율 비교

3 상위권 학생 분석

상위 10% 학생의 특징 분석,
과목별 성적 분포

4 과목별 난이도 분석

과목별 평균, 표준편차를 통해
난이도 파악

✓ 리포트 구성 예시

1. 개요
분석 주제 및 목적 소개
2. 데이터 탐색
데이터 요약 및 기초 통계
3. 분석 결과
표, 그래프를 활용한 분석 결과 제시
4. 인사이트
분석 결과에 대한 해석 및 결론
5. 느낀 점
이번 미션을 통해 배운 점 정리

★ 보고서 예시 (참고)

학과별 평균 Python 점수

학과	평균 Python
AI융합	94.2
데이터사이언스	91.3
컴퓨터공학	89.1
인공지능	88.7
...	...

성별 과목별 평균 점수



성적 분포 (Python 점수)



💡 인사이트 예시

- AI융합 학과의 Python 평균 점수가 가장 높았으며, 데이터분석 과목에서도 강세를 보임.
- 남학생이 전반적으로 높은 점수를 보였으나, 데이터분석 과목은 여학생의 평균이 더 높음.
- Python 점수는 70~89점 구간의 학생이 가장 많아, 상위권과 하위권으로 양극화되는 경향을 보임.

✓ 체크 포인트

- ☐ 다양한 Pandas 기능을 사용했나요?
- ☐ 데이터를 정확히 분석했나요?
- ☐ 시각화가 적절하게 활용되었나요?
- ☐ 인사이트가 명확하게 작성되었나요?
- ☐ 코도와 결과가 논리적으로 연결되었나요?

🚀 추가 도전 (선택)

- ★ 상관관계 히트맵 그리기
- ★ 회귀분석으로 점수 예측해보기
- ★ 이상화(특이값) 탐지 및 분석
- ★ 대시보드 형태로 시각화 확장하기

💡 팁

- 막히면 실습 노트를 다시 참고하세요.
- 다양한 그래프를 시도해 보세요!
- 질문은 언제든지 환영합니다 😊



실습 시간 : 20분

주어진 시간 안에 자유롭게 분석하고 리포트를 완성해 보세요!



스스로 해결해 보세요!

지금까지 배운 내용을 종합하여 창의적인 분석 리포트를 만들어 봅시다.



여러분의 인사이트가 빛납니다!

데이터는 답을 알고 있습니다.
여러분의 분석이 새로운 가치를 만듭니다!



실습 19

오늘 실습 정리

Pandas와 DataFrame 시작하기



★ 핵심 기능 요약

1 데이터 로드

CSV 파일을 읽어
DataFrame 생성

```
import pandas as pd
df = pd.read_csv('student_master.csv')
df.head()
```

2 데이터 확인

데이터 구조와
기본 정보 확인

```
df.head()
df.info()
df.describe()
```

3 데이터 선택

열(컬럼) 선택 및
행 인덱싱

```
df['국어']
df[['이름', '국어', '수학']]
df.iloc[0:5]
```

4 조건 필터링

조건을 만족하는
데이터 추출

```
df[df['수학'] >= 90]
df[(df['국어'] >= 80) &
    (df['영어'] >= 80)]
```

5 정렬

특정 기준으로
데이터 정렬

```
df.sort_values('수학',
               ascending=False)
df.sort_values(['학년', '국어'])
```

6 그룹별 분석

그룹별 집계 및
요약 통계

```
df.groupby('학과').agg({
    '수학': 'mean',
    '영어': 'mean'})
```

7 결측치 처리

결측치 확인 및
처리

```
df.isnull().sum()
df.fillna(df.mean(numeric_only=True),
          inplace=True)
```

8 저장 및 불러오기

데이터를 파일로 저장하고
다시 불러오기

```
df.to_csv('output.csv', index=False)
df.to_excel('output.xlsx', index=False)
df2 = pd.read_csv('output.csv')
```

9 고급 필터링/정렬

여러 조건 결합 및
다중 정렬

```
df[(df['학년']==2) & (df['수학']>=85)]
df.sort_values(['학년', '수학'],
               ascending=[True, False])
```

10 피벗 테이블

데이터를 다양한 기준으로
요약하여 재구성

```
pd.pivot_table(df,
               values='수학', index='학과',
               columns='학년', aggfunc='mean')
```

11 데이터 시각화

막대, 선 차트, 산점도 등으로
데이터 시각화

```
df.groupby('학과')['수학'].mean().plot(kind='bar')
df['수학'].hist()
df.plot.scatter(x='수학', y='영어')
```

★ 오늘 배운 내용 한눈에 보기

- ✓ CSV 파일을 읽고 DataFrame을 생성할 수 있다.
- ✓ DataFrame의 구조와 기본 정보를 확인할 수 있다.
- ✓ 행과 열을 선택하고 조건에 맞는 데이터를 추출할 수 있다.
- ✓ 조건 필터링을 통해 원하는 데이터를 추출할 수 있다.
- ✓ 데이터를 정렬하여 분석에 활용할 수 있다.
- ✓ 그룹별 집계를 통해 데이터를 요약하고 통계할 수 있다.
- ✓ 결측치를 확인하고 처리할 수 있다.
- ✓ 데이터를 저장하고 불러올 수 있다.
- ✓ 고급 필터링과 정렬을 활용할 수 있다.
- ✓ 피벗 테이블을 만들어 다양한 기준으로 요약할 수 있다.
- ✓ 시각화를 통해 데이터를 직관적으로 이해할 수 있다.

👍 오늘 실습을 통해 얻은 능력

- Pandas와 DataFrame의 핵심 기능을 익혔다.
- 데이터를 탐색하고 전처리하는 기본 능력을 갖추었다.
- 그룹화와 요약을 통해 데이터를 구조적으로 이해했다.
- 시각화를 통해 데이터의 패턴을 직관적으로 파악했다.
- 다양한 기능을 조합하여 실제 데이터 분석의 기초를 다졌다.



💡 핵심 메시지

"DataFrame은
데이터 분석의 출발점입니다."

📋 오늘 실습 흐름



🚀 다음 시간 예고

데이터 전처리와 데이터 정제

- 결측치 처리
- 이상치 탐색
- 데이터 변환
- 데이터 통합



오늘 실습 종료! Pandas와 DataFrame의 기초 기능을 탄탄히 익혔습니다. 다음 시간도 함께 힘내요!





다음 시간 주제 : 데이터 전처리와 데이터 정제

분석에 적합한 **깨끗한 데이터**를 만들기 위한 다양한 기법을 배웁니다.



핵심 메시지

좋은 분석은 좋은 데이터에서 시작됩니다.
전처리를 통해 데이터를 정제하고
분석의 **정확도**와 **신뢰도**를 높여봅시다!

1 결측치 처리



- 결측치의 종류와 원인
- 결측치 탐색 방법
- 결측치 처리 방법
 - 삭제 (dropna)
 - 대체 (fillna)
 - 보간 (interpolate)

결측치를 효과적으로 처리하여
데이터의 완성도를 높입니다.

2 이상치 탐색



- 이상치란?
- 이상치 탐색 방법
 - 기술 통계 활용
 - IQR, Z-score
 - 시각화(박스플롯 등)
- 이상치 처리 방법

이상치를 탐색하고 적절히 처리하여
분석의 신뢰성을 확보합니다.

3 데이터 변환



- 스케일링(Scaling)
 - 표준화(Standardization)
 - 정규화(Normalization)
- 인코딩(Encoding)
 - 레이블 인코딩
 - 원-핫 인코딩
- 로그 변환, 범주형 변환 등

데이터를 분석과 모델링에 적합한
형태로 변환합니다.

4 데이터 통합



- 데이터 병합(merge)
- 데이터 연결(concat)
- 조인 종류
 - inner, left, right, outer
- 키(key) 활용

여러 데이터 소스를 통합하여
분석에 필요한 데이터를 구성합니다.

✓ 다음 시간 준비하기

- ✓ 오늘 실습 내용을 복습하고 질문을 정리해 오세요.
- ✓ 데이터 전처리가 왜 중요한지 생각해 보세요.
- ✓ CSV 파일을 하나 더 준비해 오면 실습에 도움이 됩니다.



학습 목표

- ✓ 결측치와 이상치를 탐색하고 처리할 수 있다.
- ✓ 데이터 변환과 인코딩 기법을 적용할 수 있다.
- ✓ 여러 데이터를 통합하여 분석 데이터를 구성할 수 있다.

다음 시간에 만나게 될 도구

